МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждения образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет информационных технологий
Кафедра программной инженерии
Специальность 6-05-0612-01 Программная инженерия
Направление специальности 6-05-0612-01 Программная инженерия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
КУРСОВОГО ПРОЕКТ:

по дисциплине «Объектно-ориентированные технологии программирования и стандарты проектирования»
Тема «Программного обеспечения для воспроизведения и организации музыкальных коллекций»

Исполнитель
студент (ка) 2 курса группы 9 Антипов Алексей Романович
(Ф.И.О.)

Руководитель работы ассистент Мущук А.Е
(учен. степень, звание, должность, подпись, Ф.И.О.)

Курсовой проект защищен с оценкой
Председатель Белодед Н.И.
(подпись)

Минск 2026

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»
Факультет информационных технологий
Кафедра программной инженерии

Утверждаю
Заведующий кафедрой ПИ
_____В.В.Смелов
подпись инициалы и фамилия
“___”_____2026г.

ЗАДАНИЕ
к курсовому проектированию
по дисциплине "Объектно-ориентированные технологии программирования и стандарты проектирования"

Специальность: 6-05-0612-01 Программная инженерия Группа:__9__
Студент: Антипов Алексей Романович
Тема: Программного обеспечения для воспроизведения и организации музыкальных коллекций.

1. Срок сдачи студентом законченной работы: "18 мая 2026 г."
2. Исходные данные к проекту:
2.1. Функционально ПС поддерживает:

- Функции администратора сервиса:
  - Поддерживать работу с базой данных;
  - Управлять учетными записями;
  - Рассматривать жалобы пользователей;
- Функции исполнителя:
  - Загружать файлы;
  - Предлагать теги;
  - Формировать альбомы;
  - Анализировать статистику прослушиваний;
- Клиент:
  - Воспроизводить аудио;
  - Искать и сортировать контента;
  - Формировать очереди;
  - Просматривать информации о музыке.

2.2. При выполнении курсового проекта необходимо использовать принципы проектирования ООП. Приложение разрабатывается под ОС Windows и представляет собой настольное приложение (desktop). Отображение, бизнес логика должны быть максимально независимы друг от друга для возможности расширения. Диаграммы вариантов использования, классов реализации задачи, взаимодействия разработать на основе UML. Язык разработки проекта – C#. Управление программой должно быть интуитивно понятным и удобным. При разработке использовать несколько наиболее подходящих шаблонов проектирования ПО. По окончании разработки необходимо создать установщик программы.

3. Содержание расчетно-пояснительной записки

- Введение
- Анализ прототипов, литературных источников и формирование требований к проектируемому программному средству
- Анализ требований к программному средству и разработка функциональных требований

- Проектирование программного средства
- Реализация программного средства
- Тестирование, проверка работоспособности и анализ полученных результатов
- Руководство по установке и использованию
- Заключение
- Список использованных источников
- Приложения

4. Форма представления выполненной курсовой работы:

- Теоретическая часть курсового проекта должны быть представлены в формате docx. Оформление записки должно быть согласно выданным правилам.
- Листинги программы представляются в приложении.
- Пояснительную записку, листинги, проект (инсталляцию проекта) необходимо загрузить диск, указанный преподавателем.

1 Календарный план

№ п/п	Наименование этапов курсового проекта	Срок выполнения этапов проекта	Примечание
1	Введение	12.03.2026	
2	Аналитический обзор литературы по теме проекта. Изучение требований, определение вариантов использования	22.03.2026	
3	Анализ и проектирование архитектуры приложения (построение диаграмм, проектирование бизнес-слоя, представления и данных)	26.03.2026	
4	Проектирование структуры базы данных. Разработка дизайна пользовательского интерфейса	05.04.2026	
5	Кодирование программного средства	25.04.2026	
6	Тестирования и отладка программного средства	01.05.2026	
7	Оформление пояснительной записки	07.05.2026	
9	Защита проекта	18.05.2026	

5. Дата выдачи задания ____12.02.2026____

Руководитель_____ *Мущук А.Н*
(подпись)

Задание принял к исполнению _____ *Антипов А.Р.*
(дата и подпись студента)

Содержание

ВВЕДЕНИЕ 5
1 Аналитический обзор прототипов и литературных источников 6
1.1 Обзор аналогичных решений 6
1.2 Требования к проектируемому программному средству 7
2 Анализ требований к программному средству и разработка требований 9
2.1 Технические средства разработки 9
2.1.1 Язык программирования C# и платформа .Net 9
2.1.2 Entity Framework 10
2.1.3 Windows Presentation Foundation (WPF) 10
2.1.4 Microsoft SQL Server 11
2.1.5 Figma 11
2.2 Архитектура системы 11
2.3 Диаграмма вариантов использования 13
3 Проектирование программного средства 14
3.1 Проектирования логической структуры базы данных 14
3.2 Проектирования структуры окон 16
3.3 Проектирование архитектуры приложения 17
3.4 UML-диаграмма классов 18
4 Реализация программного средства 22
4.1 Основные классы и методы программного средства 22
4.2 Классы для реализации паттерна MVVM 22
4.3 Классы для реализации паттерна Command 24
4.4 Классы для реализации взаимодействия с базой данных 25
4.5 Методы для работы с файлами 27
4.2 Методы для регистрации и авторизации пользователей 28
5 Тестирование, проверка работоспособности и анализ полученных результатов 30
5.1 Тестировании регистрации и авторизации 30
5.2 Тестирование работы с музыкальным плеером 33
5.3 Тестирование создания и редактирования плейлистов 35
5.4 Тестирование поиска 38
5.5 Тестирование страницы избранного 39
5.6 Тестирование студии исполнителя 40
5.7 Тестирование жалоб на треки 41
5.8 Тестирование профиля пользователя 42
5.9 Тестирование панели администратора 43

5.10 Тестирование альбомов и страниц исполнителей 46
6 Руководство по использованию 48
ЗАКЛЮЧЕНИЕ 53
Список использованных источников 54
ПРИЛОЖЕНИЕ А 55
ПРИЛОЖЕНИЕ Б 56
ПРИЛОЖЕНИЕ В 62
ПРИЛОЖЕНИЕ Г 63
ПРИЛОЖЕНИЕ Д 69
ПРИЛОЖЕНИЕ Е 71
ПРИЛОЖЕНИЕ Ж 74

ВВЕДЕНИЕ

Музыкальные стриминговые сервисы занимают одно из центральных мест в современной цифровой индустрии. Ежедневно миллионы пользователей обращаются к платформам для воспроизведения, организации и поиска музыкального контента. С развитием потребностей пользователей возрастают и требования к функциональности, удобству интерфейса и производительности подобных приложений.

Тема данного курсового проекта — «Программное обеспечение для воспроизведения и организации музыкальных коллекций» — охватывает как технические, так и пользовательские аспекты создания музыкального плеера с расширенными возможностями. Актуальность разработки обусловлена устойчивым ростом объёма цифрового музыкального контента, а также потребностью пользователей в удобных инструментах для его организации, поиска и прослушивания.

Целью данного курсового проектирования является проектирование и реализация настольного программного средства «Rewind» — приложения для воспроизведения и организации музыкальных коллекций. Приложение поддерживает три роли пользователей: слушатель, исполнитель и администратор, каждая из которых имеет собственный набор функциональных возможностей.

В основе проектирования программного средства лежат следующие принципы:

— разделение ответственности между слоями представления, логики и данных с использованием архитектурного паттерна MVVM;

— интуитивно понятный пользовательский интерфейс, обеспечивающий комфортную работу как слушателям, так и исполнителям;

— гибкая ролевая модель, позволяющая разграничить доступ к функциям приложения;

— надёжное и масштабируемое хранение данных с использованием СУБД PostgreSQL.

В результате проектирования и реализации приложения «Rewind» пользователи получают инструмент, обеспечивающий полноценное воспроизведение музыки, управление плейлистами и избранными треками, поиск и фильтрацию контента по жанрам, а также административный контроль над публикуемым контентом.

1 Аналитический обзор прототипов и литературных источников

1.1 Обзор аналогичных решений

Одним из первых этапов в создании программного продукта является анализ прототипов и литературных источников. На сегодняшний день рынок музыкальных стриминговых сервисов и медиаплееров представлен множеством решений с различным набором функциональных возможностей. Рассмотрим наиболее распространённые из них.

Первый аналог — «Spotify». Spotify — это ведущий музыкальный стриминговый сервис, предоставляющий пользователям доступ к обширной библиотеке треков, подкастов и аудиокниг. Сервис доступен в виде веб-приложения, а также десктопного и мобильного клиентов для всех основных платформ. Spotify предлагает персонализированные рекомендации на основе алгоритмов машинного обучения, создание плейлистов, управление избранными треками и совместное прослушивание.

К ключевым особенностям Spotify относятся: умные рекомендательные алгоритмы, поддержка офлайн-прослушивания (в платной версии), развитые социальные функции, кроссплатформенная синхронизация. Вместе с тем сервис является облачным, не предоставляет возможности загрузки собственного контента рядовым пользователям и требует постоянного подключения к интернету в бесплатной версии.

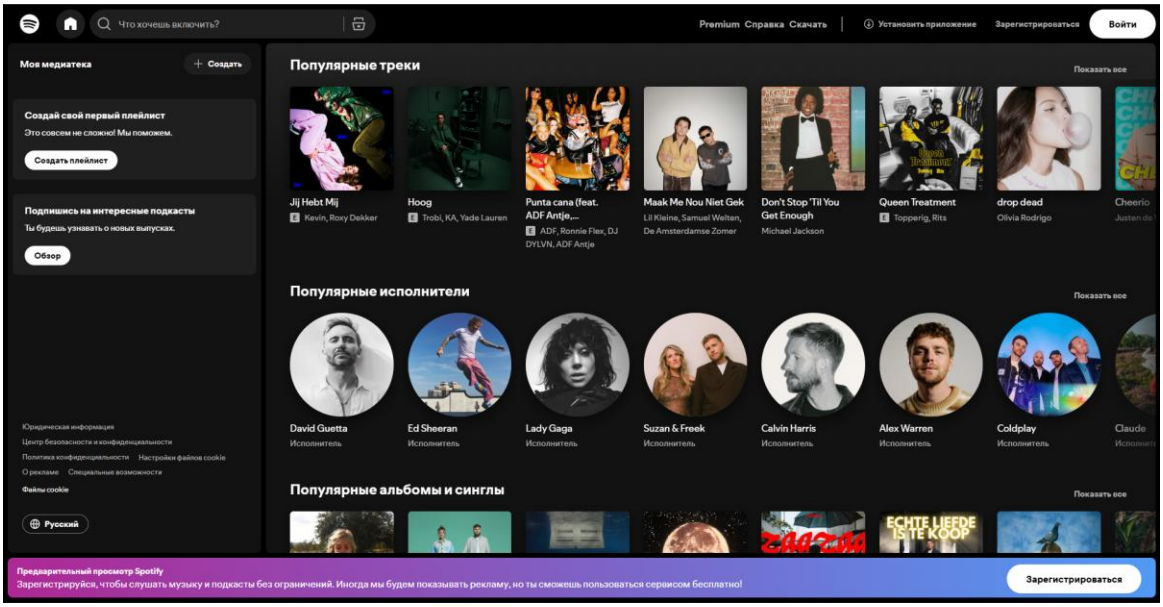


Рисунок 1.1 — Интерфейс приложения Spotify

Второй аналог — «AIMP». AIMP — это бесплатный локальный медиаплеер для Windows с поддержкой большого количества аудиоформатов. Он ориентирован на воспроизведение локальных музыкальных коллекций, поддерживает эквалайзер, плейлисты, визуализатор и управление

библиотекой. В отличие от стриминговых сервисов, AIMP не предполагает облачного хранилища и социальных функций, однако обеспечивает высокое качество воспроизведения и широкие возможности настройки звука.



Рисунок 1.2 — Интерфейс приложения AIMP

Также в качестве аналогов можно выделить следующие приложения: Apple Music, YouTube Music, VK Music, SoundCloud. Каждый из перечисленных аналогов обладает уникальным набором функций и ориентирован на определённую аудиторию. Все рассмотренные аналоги имеют схожую тематику и функционал, но у каждого есть отличительные особенности, что и делает каждое из них уникальным.

1.2 Требования к проектируемому программному средству

Основываясь на анализе аналогичных решений из раздела 1.1, проектируемое программное средство должно включать следующие характеристики и функциональные возможности.

Приложение должно обеспечивать воспроизведение аудиофайлов с полноценным управлением (воспроизведение, пауза, перемотка, переход к следующему и предыдущему треку). Должна быть реализована регулировка громкости, режимы повтора и перемешивания очереди воспроизведения.

Пользовательский интерфейс должен быть визуально привлекательным и интуитивно понятным. Необходима поддержка нескольких тем оформления для обеспечения персонализации приложения.

Должна быть реализована ролевая модель с тремя типами пользователей:

— слушатель — может воспроизводить треки, управлять плейлистами, добавлять треки в избранное, искать и фильтровать контент;

— исполнитель — обладает возможностями слушателя, а также может загружать треки и формировать альбомы;

— администратор — управляет пользователями, модерирует заявки на регистрацию исполнителей, рассматривает жалобы и публикует треки.

Функциональность поиска и фильтрации по жанрам и исполнителям является обязательной. Пользователь должен иметь возможность формировать собственные плейлисты (публичные и приватные), управлять избранными треками, просматривать профили исполнителей и их альбомы.

Хранение всех данных должно осуществляться в реляционной базе данных. Пароли пользователей должны храниться в хешированном виде с использованием современных криптографических алгоритмов.

Резюмируя, проектируемое программное средство должно сочетать функциональность локального медиаплеера и стримингового сервиса, обеспечивая удобное воспроизведение, организацию и поиск музыкального контента в рамках настольного приложения для Windows.

2 Анализ требований к программному средству и разработка требований

Анализ требований — это процесс сбора требований к программному обеспечению, их систематизации, документирования, анализа, выявления противоречий, неполноты и разрешения конфликтов в процессе разработки программного обеспечения.

Целью анализа требований является получение максимума информации о специфике задачи, уточнение рамок проекта, оценка возможных рисков. На этом этапе происходит идентификация принципиальных требований методологического и технологического характера, формулируются цели и задачи проекта, а также определяются критические факторы успеха.

Данный раздел начнём с технических средств, которые использовались для разработки программного средства, после чего перейдём к архитектуре системы и диаграмме вариантов использования.

2.1 Технические средства разработки

Для создания программного средства «Rewind» были выбраны современные технологии и инструменты, обеспечивающие высокое качество и удобство разработки. В процессе создания приложения использовался язык программирования C# и платформа .NET 10, фреймворк для построения интерфейса Windows Presentation Foundation (WPF), система управления базами данных PostgreSQL и объектно-реляционный маппер Entity Framework Core. Для проектирования макетов применялся инструмент Figma.

2.1.1 Язык программирования C# и платформа .Net

C# — это строго типизированный объектно-ориентированный язык программирования, разработанный компанией Microsoft в конце 1990-х годов. Язык создавался под руководством Андерса Хейлсберга и объединил в себе лучшие черты Java, C++ и Delphi. C# является основным языком разработки для экосистемы .NET и активно развивается: в современных версиях поддерживаются паттерн-матчинг, записи (records), nullable-типы, асинхронные потоки и другие возможности.

Платформа .NET представляет собой свободно распространяемую кросс-платформенную среду выполнения с богатой стандартной библиотекой классов. Для данного проекта использовалась версия .NET 10, которая обеспечивает высокую производительность, поддержку WPF для Windows и современные языковые возможности C#. .NET предоставляет автоматическую сборку мусора, механизм рефлексии, LINQ, асинхронное программирование и многое другое, что значительно упрощает разработку сложных приложений.

Таким образом, C# и платформа .NET представляют собой мощный союз, обеспечивающий разработчикам средства и среду для создания современных и высокоэффективных приложений.

2.1.2 Entity Framework

Entity Framework Core (EF Core) — это современная объектно-реляционная система отображения (ORM) для .NET, разработанная компанией Microsoft. EF Core позволяет работать с базой данных, используя объекты C# вместо написания SQL-запросов вручную. Основной концепцией EF Core является контекст данных (DbContext) — специальный класс, предоставляющий доступ к таблицам базы данных через типизированные коллекции DbSet<T>.

В данном проекте использовался подход Code-First, при котором структура базы данных создаётся на основе классов-сущностей C#. Для взаимодействия с СУБД PostgreSQL применяется провайдер Npgsql.EntityFrameworkCore.PostgreSQL. EF Core поддерживает LINQ-запросы, явную загрузку связанных сущностей (Include) и миграции для управления схемой базы данных.

Благодаря Entity Framework Core разработка приложения «Rewind» с точки зрения работы с данными становится эффективной и наглядной: таблицы базы данных представляются как обычные коллекции объектов C#, а изменения в данных автоматически транслируются в соответствующие SQL-инструкции.

2.1.3 Windows Presentation Foundation (WPF)

Windows Presentation Foundation (WPF) — это подсистема пользовательского интерфейса, входящая в состав .NET и предназначенная для разработки настольных приложений Windows. WPF использует язык декларативного описания интерфейса XAML, основанный на XML. Это позволяет чётко разделить логику и представление, что является фундаментальным принципом архитектуры MVVM.

WPF обеспечивает аппаратно-ускоренный рендеринг через DirectX, поддержку анимаций, привязки данных (Data Binding), стили и шаблоны элементов управления. Использование ресурсных словарей (ResourceDictionary) позволяет реализовать систему тем оформления, применяемую в данном проекте для переключения цветовых схем интерфейса без перезапуска приложения.

В итоге WPF представляет собой гармоничное слияние технологии и дизайна, где разработчики могут воплотить свои творческие решения в современные настольные приложения, которые не только функциональны, но и визуально привлекательны.

2.1.4 Microsoft SQL Server

PostgreSQL — это высокопроизводительная объектно-реляционная система управления базами данных с открытым исходным кодом. Она отличается надёжностью, соответствием стандартам SQL, богатым набором типов данных и широкими возможностями расширения. PostgreSQL является одной из наиболее популярных СУБД в мире и активно применяется как в небольших проектах, так и в крупных производственных системах.

В данном проекте PostgreSQL версии 17 запускается в Docker-контейнере с использованием docker-compose, что обеспечивает простоту развёртывания и воспроизводимость среды. База данных rewinddb хранит всю информацию о пользователях, треках, плейлистах, альбомах, статистике и других сущностях приложения.

2.1.5 Figma

Figma — инновационный инструмент для дизайна и прототипирования пользовательского интерфейса, работающий в браузере. Одним из главных преимуществ Figma является возможность совместной работы в режиме реального времени. Участники команды могут одновременно работать над проектом, видеть изменения других участников и общаться во время работы.

При разработке приложения «Rewind» Figma использовалась для проектирования макетов всех экранов приложения до начала реализации. Это позволило заранее продумать навигационную структуру, цветовые схемы и расположение элементов управления, существенно ускорив последующую разработку.

2.2 Архитектура системы

Для данного курсового проекта был выбран способ разработки программных модулей, основанный на архитектуре настольного клиентского приложения. Приложение состоит из двух основных частей:

— Клиентская часть. В данном модуле осуществляется взаимодействие пользователя с приложением посредством графического интерфейса. Пользователь получает доступ к музыкальному плееру, плейлистам, поиску, профилю и другим функциям в соответствии с присвоенной ему ролью.

— База данных. В базе данных приложения хранится информация о пользователях, треках, плейлистах, альбомах, статистике прослушиваний, избранном, жалобах и прочих данных системы.

Различают три уровня требований к проекту:

— бизнес-требования;

— пользовательские требования;

— функциональные требования.

Бизнес-требования определяют высокоуровневые цели системы. К ним относятся: простота и привлекательность интерфейса; использование принципов ООП; применение архитектурных шаблонов проектирования; использование СУБД для хранения данных; поддержка нескольких ролей пользователей.

Пользовательские требования описывают цели и задачи, которые пользователям позволит решить система. Слушатель сможет:

— регистрироваться и входить в приложение;

— воспроизводить треки, управлять очередью воспроизведения;

— добавлять треки в избранное;

— создавать и управлять плейлистами (публичными и приватными);

— искать треки по названию, исполнителю и жанру;

— просматривать профили исполнителей и их альбомы;

— изменять тему оформления приложения;

— отправлять жалобы на треки.

Исполнитель, помимо функций слушателя, сможет:

— загружать аудиофайлы с обложками и жанровыми тегами;

— создавать и редактировать альбомы;

— просматривать статистику прослушиваний своих треков.

Администратор сможет:

— управлять учётными записями пользователей;

— рассматривать заявки на регистрацию исполнителей и одобрять или отклонять их;

— модерировать загружаемые треки (принимать или отклонять публикацию);

— рассматривать жалобы пользователей на треки.

После проведения анализа были сформулированы следующие функциональные требования:

— вся информация должна храниться в реляционной базе данных;

— приложение должно производить валидацию вводимых данных (формат email, сложность пароля, уникальность никнейма);

— приложение должно корректно обрабатывать исключительные ситуации и отображать понятные пользователю сообщения об ошибках;

— пароли пользователей должны храниться в хешированном виде;

— приложение должно поддерживать смену темы оформления без перезапуска;

— музыкальный плеер должен поддерживать воспроизведение, паузу, перемотку, регулировку громкости, режимы повтора и перемешивания.

Таким образом, был проведён тщательный анализ требований к программному средству, который позволил разработать полный список функциональных требований. Разработка данной программной системы должна проводиться в соответствии со сформированным списком.

2.3 Диаграмма вариантов использования

Диаграмма вариантов использования (Use Case Diagram) является графическим средством моделирования, которое позволяет представить функциональность системы с точки зрения её пользователей. Она помогает идентифицировать и описать различные сценарии использования системы, включая взаимодействие между актёрами и самой системой.

Диаграмма вариантов использования состоит из актёров, вариантов использования и связей между ними. Актёры представляют различных пользователей или внешние системы, взаимодействующие с программой. Варианты использования представляют собой функциональные возможности системы. В данном проекте определены три актёра: Слушатель, Исполнитель и Администратор.

Преимущества использования диаграммы вариантов использования включают:

— Визуализацию функциональности системы: диаграмма помогает увидеть все возможные сценарии использования и взаимодействие с пользователями;

— Определение требований: диаграмма помогает идентифицировать и описать функциональные требования к системе на основе сценариев использования;

— Улучшение коммуникации: диаграмма служит средством коммуникации между разработчиками и заинтересованными сторонами.

UML-диаграмма вариантов использования для программного средства «Rewind» представлена в приложении А.

3 Проектирование программного средства

3.1 Проектирования логической структуры базы данных

Для хранения данных приложения используется система управления реляционными базами данных PostgreSQL 17. База данных развёртывается в Docker-контейнере с использованием docker-compose, что обеспечивает воспроизводимость и лёгкость настройки окружения.

База данных — это совокупность данных, организованных по определённым правилам, предусматривающим общие принципы описания, хранения и манипулирования данными независимо от прикладных программ. Система управления базами данных — совокупность программ и языковых средств, предназначенных для управления данными в базе данных и обеспечения взаимодействия её с прикладными программами.

База данных данного курсового проекта состоит из 18 таблиц. Схема базы данных изображена на рисунке 4.1.

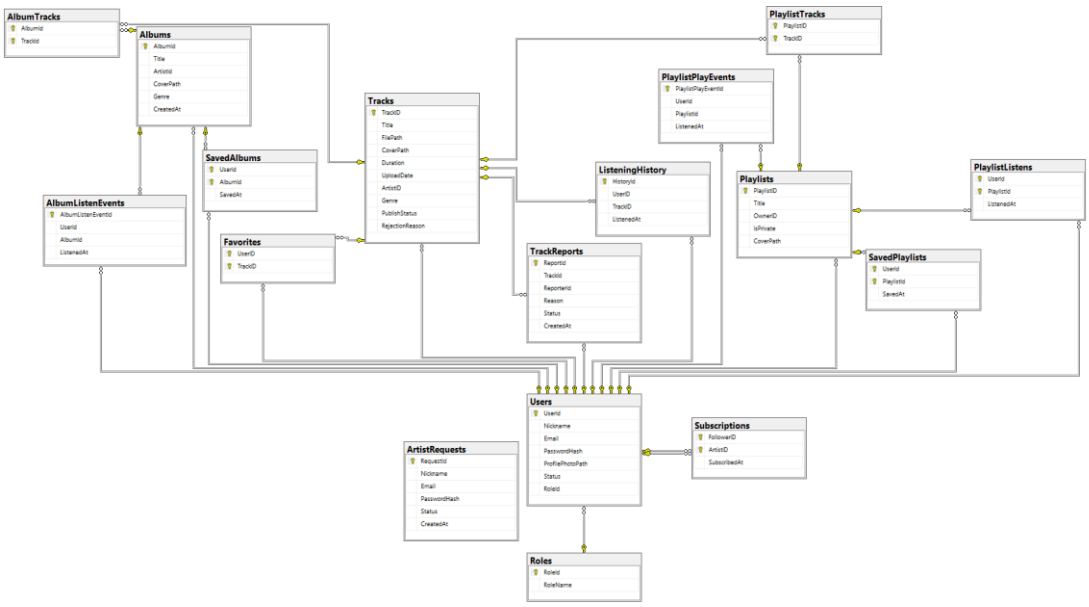


Рисунок 3.1 — Структура базы данных

Остановимся подробнее на основных таблицах. Центральным звеном подсистемы аутентификации и авторизации является таблица Users. Она служит единым репозиторием для хранения учетных данных и атрибутов профиля каждого зарегистрированного участника системы. Структура таблицы спроектирована таким образом, чтобы обеспечить баланс между информативностью профиля и требованиями информационной безопасности. Таблица Users содержит информацию обо всех зарегистрированных пользователях: их никнейм, email, хеш пароля, путь к фотографии профиля, статус и идентификатор роли (внешний ключ на таблицу Roles). Проектирование таблицы **Users** с учетом внешнего ключа на таблицу ролей

позволяет реализовать гибкую систему разграничения прав. На рисунке 3.2 представлена подробная информация об этой таблице.

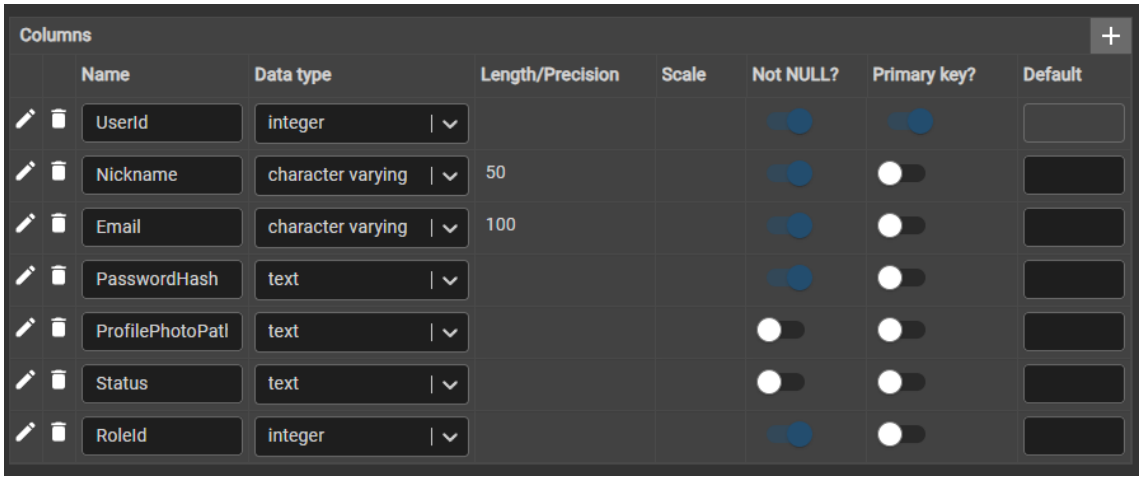


Рисунок 3.2 — Таблица Users

Таблица Tracks хранит информацию о треках: название, путь к файлу, путь к обложке, продолжительность, жанр, статус публикации и идентификатор исполнителя. На рисунке 3.3 представлена подробная информация об этой таблице.

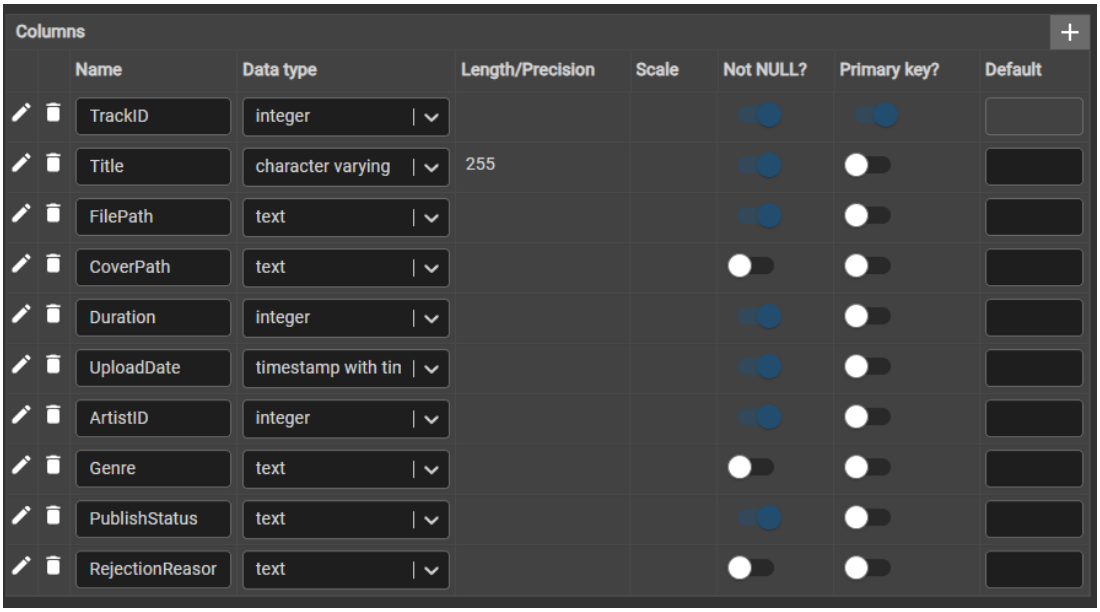


Рисунок 3.3 — Таблица Tracks

Следующим важным элементом базы данных является таблица Playlists. Она предназначена для структурирования пользовательского контента и позволяет группировать аудиотреки в тематические подборки. Данная таблица не только хранит метаданные самих списков воспроизведения, но и отвечает за управление доступом к ним. Таблица Playlists содержит информацию о плейлистах пользователей, включая признак приватности. Наличие поля

Privacy Status позволяет реализовать гибкую логику отображения контента. На уровне SQL-запросов это обеспечивает фильтрацию: при просмотре чужого профиля система автоматически исключает записи, помеченные как приватные. На рисунке 3.4 представлена подробная информация об этой таблице.

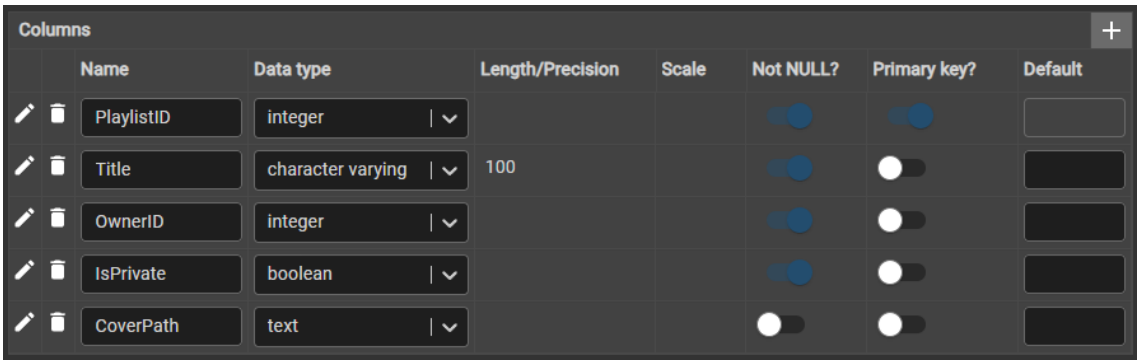


Рисунок 3.4 — Таблица Playlists

Таблица Favorites хранит информацию об избранных треках пользователей. Таблица ArtistRequests используется для хранения заявок на регистрацию исполнителей, поданных на рассмотрение администратору. Таблица TrackReports хранит жалобы пользователей на треки. Таблица Albums хранит альбомы исполнителей, а таблица AlbumTracks реализует связь альбомов с треками. Таблица Roles определяет возможные роли пользователей: администратор, исполнитель и слушатель. Таблица PlaylistTracks реализует связь многие-ко-многим между плейлистами и треками. Таблица Statistics хранит статистику по каждому треку (количество прослушиваний и лайков) и связана с Tracks отношением один-к-одному.

Между таблицами реализованы связи один-ко-многим и многие-ко-многим. Связь один-ко-многим означает, что несколько строк дочерней таблицы ссылаются на одну строку родительской таблицы. Связь многие-ко-многим реализуется через промежуточную таблицу-связку (PlaylistTracks, AlbumTracks, SavedAlbums).

При работе с базой данных применяется Entity Framework Core с провайдером Npgsql. В проектировании базы данных использовался подход Code-First: структура базы данных создаётся на основе написанных классов C#. Это даёт полный контроль над структурой базы данных и позволяет гибко адаптироваться к изменяющимся требованиям.

3.2 Проектирования структуры окон

Приложение включает в себя два основных окна: Registration (регистрация и авторизация) и MainWindow (главное окно для слушателей и исполнителей), а также панель администратора AdminPanel и вспомогательные окна: IslandWindow (плавающий мини-плеер), NowPlaying

(полноэкранный режим воспроизведения) и ReportTrackDialog (диалог жалобы на трек).

Стартовым окном является окно регистрации и авторизации (Registration). Пользователь выбирает роль (Слушатель, Исполнитель, Администратор) и либо вводит данные для входа, либо заполняет форму регистрации. При регистрации исполнителя заявка отправляется на рассмотрение администратору, а сам пользователь получает уведомление об ожидании одобрения.

В зависимости от выбранной роли, пользователь переходит либо в главное окно приложения (MainWindow), либо в панель администратора (AdminPanel).

Главное окно содержит боковую навигационную панель со следующими пунктами:

— Главная — лента треков дня и популярных плейлистов с рекомендованным треком;

— Плейлисты — управление собственными и сохранёнными плейлистами;

— Избранное — просмотр понравившихся треков с фильтрацией и сортировкой;

— Поиск — поиск треков по названию, исполнителю и жанру;

— Профиль — настройки аккаунта, смена темы;

— Студия (только для исполнителей) — загрузка треков и управление альбомами.

В нижней части главного окна постоянно отображается панель плеера (PlayerBar) с элементами управления воспроизведением.

Административная панель содержит вкладки:

— Обзор — сводная статистика системы с графиками;

— Пользователи — список всех пользователей системы;

— Заявки исполнителей — рассмотрение новых заявок на регистрацию;

— Загрузки треков — модерация треков, ожидающих публикации;

— Жалобы — рассмотрение жалоб пользователей на треки.

Таким образом, для разных ролей пользователей предусмотрен разный функционал и, соответственно, разные окна и страницы.

3.3 Проектирование архитектуры приложения

Архитектура программного обеспечения представляет собой совокупность ключевых решений, касающихся организации программной системы. Она включает выбор структурных элементов и их интерфейсов, определение их взаимодействия, а также объединение этих элементов в более крупные системы.

Для обеспечения чистоты кода и разделения зон ответственности в проекте был выбран архитектурный паттерн Model-View-ViewModel (MVVM). Его использование позволяет изолировать бизнес-логику от

интерфейса пользователя, что значительно упрощает поддержку и масштабирование приложения. На диаграмме, рисунке 3.3, наглядно продемонстрирована иерархия и связи между тремя ключевыми уровнями системы.

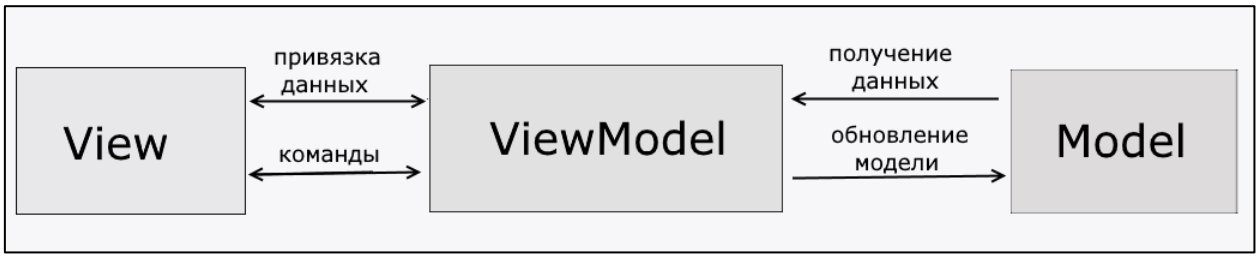


Рисунок 3.3 — Структура паттерна MVVM

Модель описывает данные, используемые в приложении. Классы-сущности (User, Track, Playlist, Album и другие) описывают структуру данных и используются Entity Framework Core для создания таблиц в базе данных. Модель не содержит логику, связанную с отображением данных и взаимодействием с визуальными элементами управления.

Представление (View) определяет визуальный интерфейс. В данном проекте представление описывается в XAML-файлах страниц и UserControl-ов. View не содержит бизнес-логики, а взаимодействует с ViewModel через привязки данных (Binding) и команды (Command).

ViewModel (модель представления) связывает модель и представление через механизм привязки данных. ViewModel реализует интерфейс INotifyPropertyChanged, благодаря которому при изменении свойств данные в представлении автоматически обновляются. ViewModel также содержит команды (объекты ICommand), выполняющие действия в ответ на события пользовательского интерфейса.

Помимо паттерна MVVM, в проекте используются следующие паттерны:

— Command (RelayCommand) — для реализации команд в ViewModel без прямой зависимости от View;

— Service Locator — для регистрации и резолвинга сервисов (навигации, диалогов, плеера), что обеспечивает слабую связанность компонентов системы.

Сервисы (NavigationService, PlayerService, DialogService) регистрируются в ServiceLocator при запуске приложения в классе App.OnStartup и доступны из любого компонента системы через статический метод ServiceLocator.Resolve<T>().

3.4 UML-диаграмма классов

Из-за значительного количества программных классов, общая диаграмма была разделена на несколько логических частей. Первая из них —

классы, необходимые для полноценной реализации паттерна Command, рисунок 3.4. В частности, классы RelayCommand и RelayCommand функционально реализуют стандартный интерфейс ICommand, инкапсулируя в себе делегаты Execute и CanExecute для управления логикой действий.

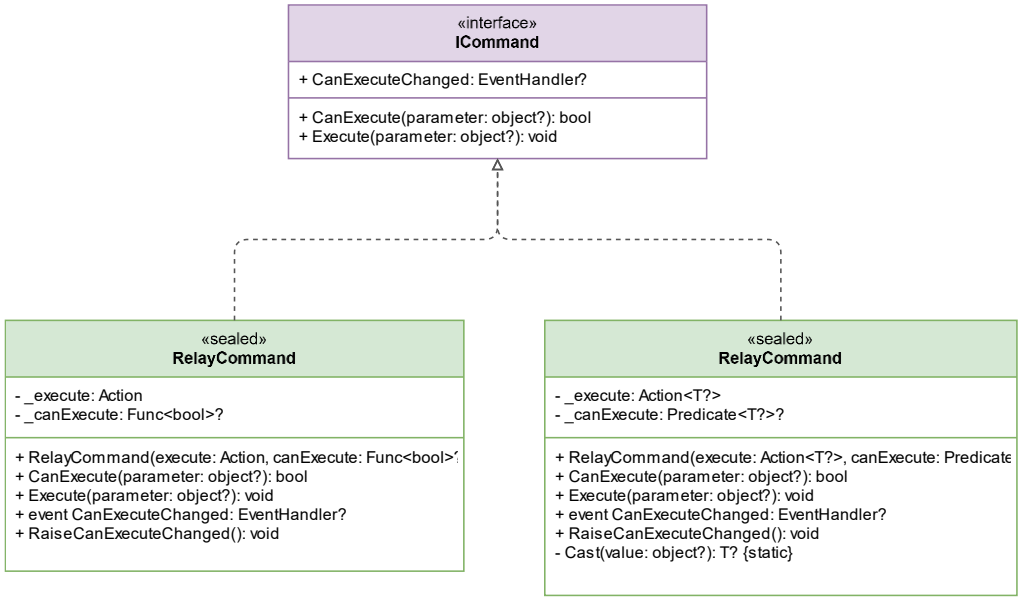


Рисунок 3.4 — UML-диаграмма классов для реализации команд

На рисунке 3.5 изображены классы ViewModelBase и ObservableObject, являющиеся базовыми классами для реализации паттерна MVVM. ViewModelBase реализует INotifyPropertyChanged и предоставляет утилитарные методы OnPropertyChanged и SetProperty.

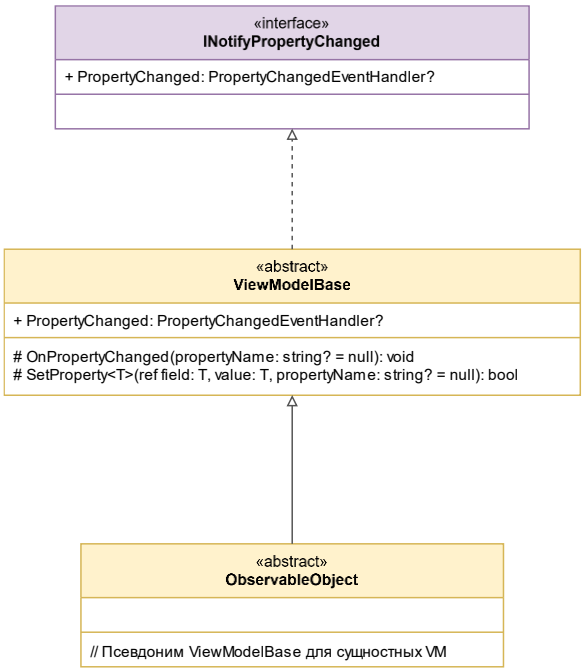


Рисунок 3.5 — UML-диаграмма базовых классов ViewModel

На рисунке 3.6 детально изображены конкретные классы ViewModel, отвечающие за функционал страниц приложения. Все они наследуются от базового класса ViewModelBase и реализуют специфическую логику для каждой части интерфейса: MainPageViewModel, SearchPageViewModel, FavoritesPageViewModel, PlaylistsPageViewModel, ProfilePageViewModel, RegistrationViewModel, а также специализированные AdminPanelViewModel, ArtistStudioViewModel и другие сопутствующие модули.

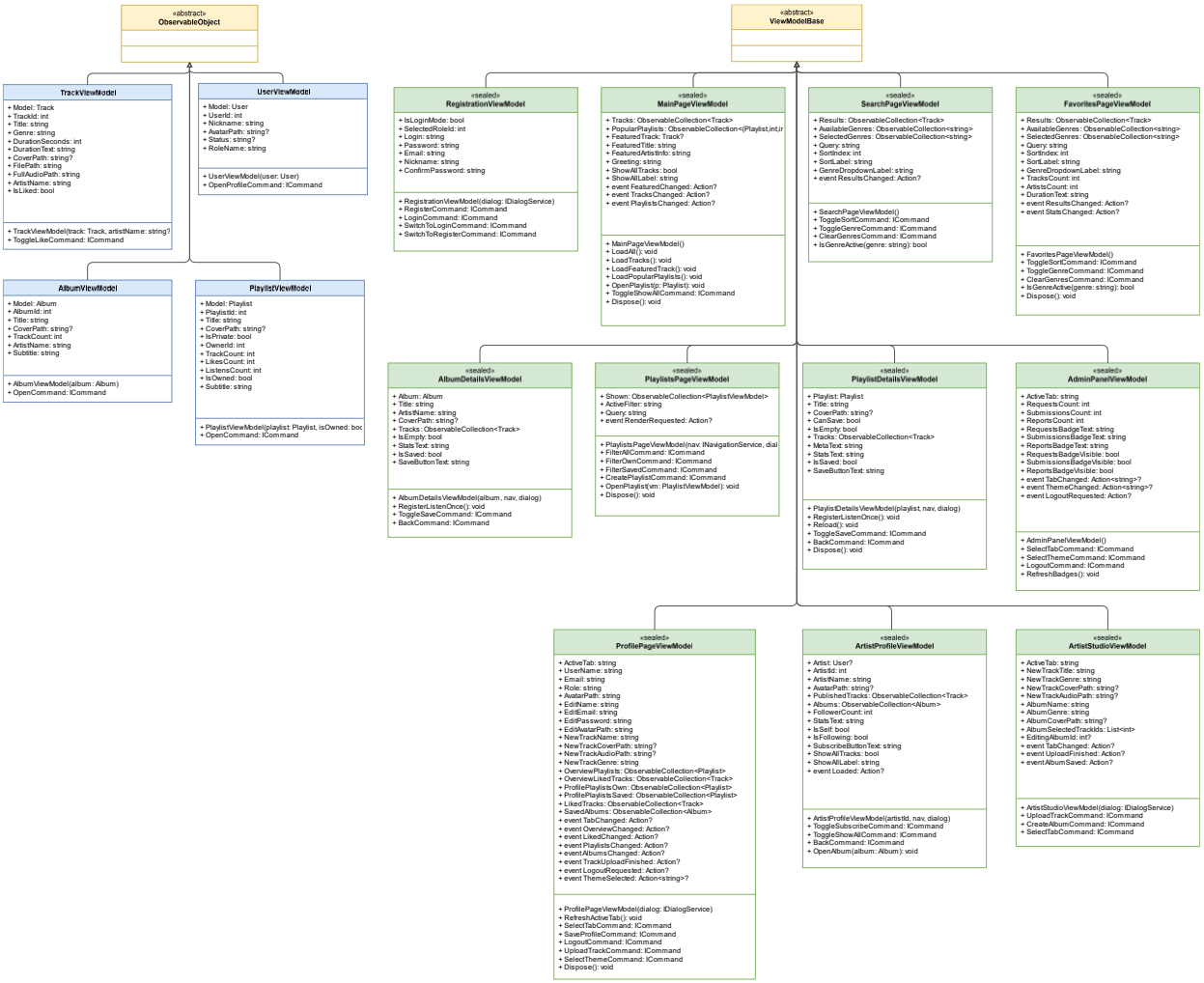


Рисунок 3.6 — UML-диаграмма классов ViewModel страниц

Дополнительно в структуре проекта выделяются специализированные классы-сущности, необходимые для взаимодействия с Entity Framework Core (такие как User, Track, Playlist, Album и другие), которые фактически представляют собой объектно-ориентированные модели таблиц из базы данных. Параллельно с ними функционируют статические сервисные классы для прямой работы с данными (TrackService, UserService, PlaylistService), полностью инкапсулирующие в себе сложную логику SQL-запросов, а также важные вспомогательные инструменты: Session для глобального управления состоянием текущего пользователя, PasswordHelper для реализации

алгоритмов безопасного хеширования паролей и FileStorage, отвечающий за корректное сохранение и поиск медиафайлов в файловой системе.

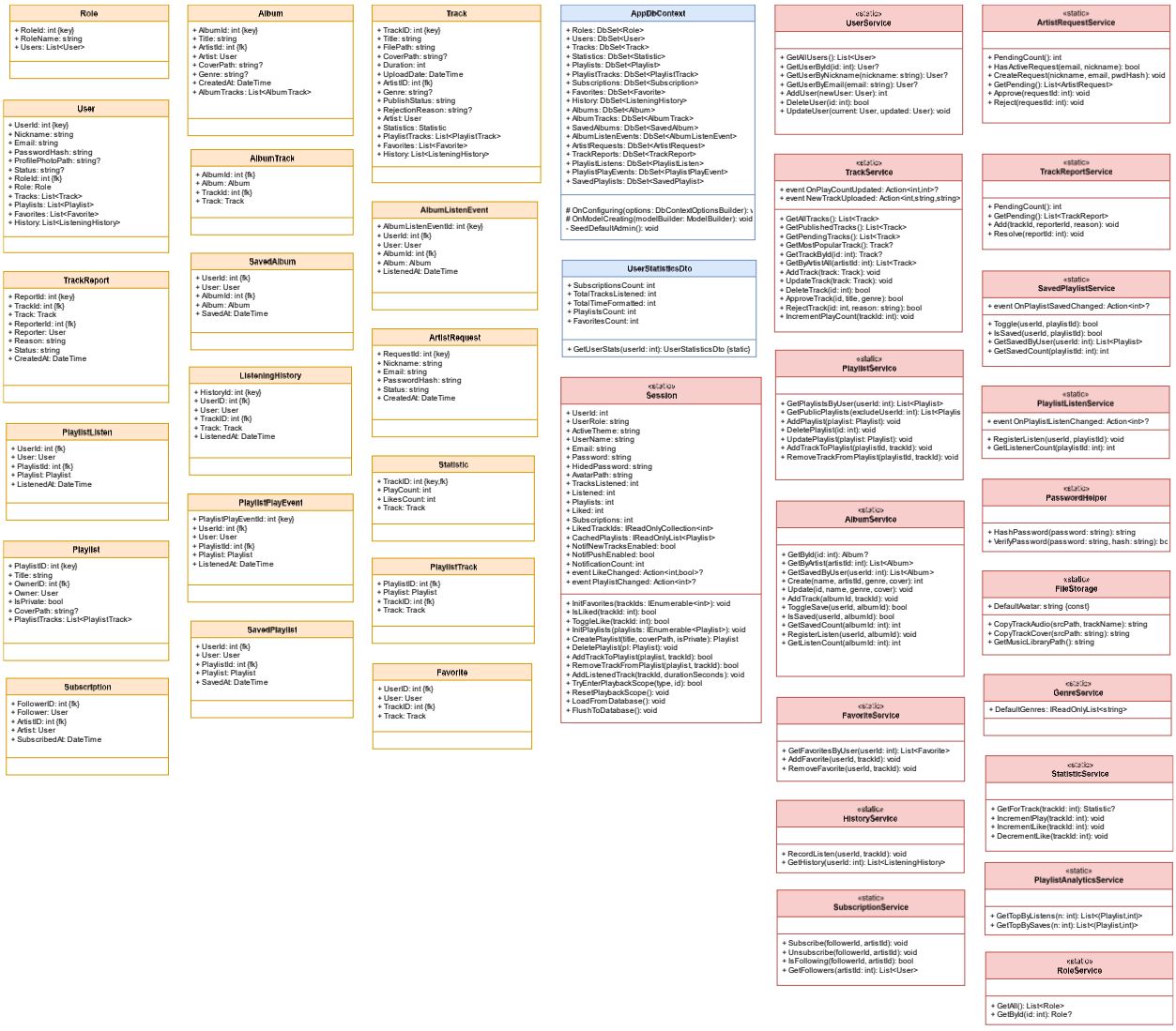


Рисунок 3.7 — UML-диаграмма классов сущностей и сервисов

Диаграмма классов позволяет определить основные классы, их свойства и методы, увидеть структуру данных и взаимосвязи между классами, обеспечивая основу для дальнейшей реализации программного средства.

4 Реализация программного средства

4.1 Основные классы и методы программного средства

В данном разделе будут рассмотрены основные классы с точки зрения архитектуры программы. Естественно, что при разработке программного средства создаётся большое количество вспомогательных классов, однако наибольший интерес с архитектурной точки зрения представляют:

— классы для реализации паттерна MVVM;
— классы для реализации паттерна Command (RelayCommand);
— классы для взаимодействия с базой данных;
— методы для работы с файлами;
— методы для регистрации и авторизации пользователей.

4.2 Классы для реализации паттерна MVVM

Как отмечалось в предыдущих разделах, при разработке приложения «Rewind» использовался архитектурный паттерн MVVM. Для его реализации применяются следующие ключевые классы: ViewModelBase, ObservableObject и конкретные ViewModel-ы страниц.

ViewModelBase — это абстрактный базовый класс для всех ViewModel. Он реализует интерфейс INotifyPropertyChanged и предоставляет два утилитарных метода: OnPropertyChanged — для генерации уведомлений об изменениях свойств, и SetProperty — для присваивания значения полю с одновременной проверкой на изменение и уведомлением. На рисунке 4.1 приведён фрагмент реализации данного класса.

```
using System.ComponentModel;
using System.Runtime.CompilerServices;

namespace Rewind.MVVM.Core
{
    // Базовый класс вьюмодел
    Ссылок: 23
    public abstract class ViewModelBase : INotifyPropertyChanged
    {
        public event PropertyChangedEventHandler? PropertyChanged;

        Ссылок: 35
        protected void OnPropertyChanged([CallerMemberName] string? propertyName = null)
            => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));

        // Присваивает новое значение и уведомляет об изменении успех — true
        Ссылок: 22
        protected bool SetProperty<T>(ref T field, T value, [CallerMemberName] string? propertyName = null)
        {
            if (EqualityComparer<T>.Default.Equals(field, value)) return false;
            field = value;
            OnPropertyChanged(propertyName);
            return true;
        }
    }
}
```

Рисунок 4.1 — Класс ViewModelBase

ObservableObject наследует ViewModelBase и используется как псевдоним для сущностных моделей, которые не являются ViewModel-ами

конкретной страницы, но нуждаются в INotifyPropertyChanged (например, карточка трека в списке).

На примере класса MainPageViewModel рассмотрим пример использования данной модели. MainPageViewModel содержит коллекции треков и плейлистов (ObservableCollection<T>), свойства с уведомлениями, команды и методы загрузки данных из базы. Дополнительно класс подписывается на события сервисов (TrackService.OnPlayCountUpdated, PlaylistListenService.OnPlaylistListenChanged) и реализует IDisposable для корректной отписки при уничтожении объекта. Фрагмент класса изображён на рисунке 4.2.

```
namespace Rewind.MVVM.ViewModels.Pages
{
    Ссылок: 4
    public sealed class MainPageViewModel : ViewModelBase, IDisposable
    {
        Ссылок: 1
        public MainPageViewModel()
        {
            Tracks = new ObservableCollection<Track>();
            PopularPlaylists = new ObservableCollection<(Playlist Playlist, int Saves, int Listens)>();

            ToggleShowAllCommand = new RelayCommand(() =>
            {
                ShowAllTracks = !ShowAllTracks;
                LoadTracks();
            });

            TrackService.OnPlayCountUpdated += OnPlayCountUpdated;
            PlaylistListenService.OnPlaylistListenChanged += OnPlaylistStatsChanged;
            SavedPlaylistService.OnPlaylistSavedChanged += OnPlaylistStatsChanged;
        }

        Ссылок: 4
        public ObservableCollection<Track> Tracks { get; }
        Ссылок: 4
        public ObservableCollection<(Playlist Playlist, int Saves, int Listens)> PopularPlaylists { get; }

        public event Action? FeaturedChanged;
        public event Action? TracksChanged;
        public event Action? PlaylistsChanged;

        private Track? _featuredTrack;
        Ссылок: 19
        public Track? FeaturedTrack
        {
            get => _featuredTrack;
            private set
            {
                if (SetProperty(ref _featuredTrack, value))
                {
                    OnPropertyChanged(nameof(FeaturedTitle));
                    OnPropertyChanged(nameof(FeaturedArtistInfo));
                    FeaturedChanged?.Invoke();
                }
            }
        }
```

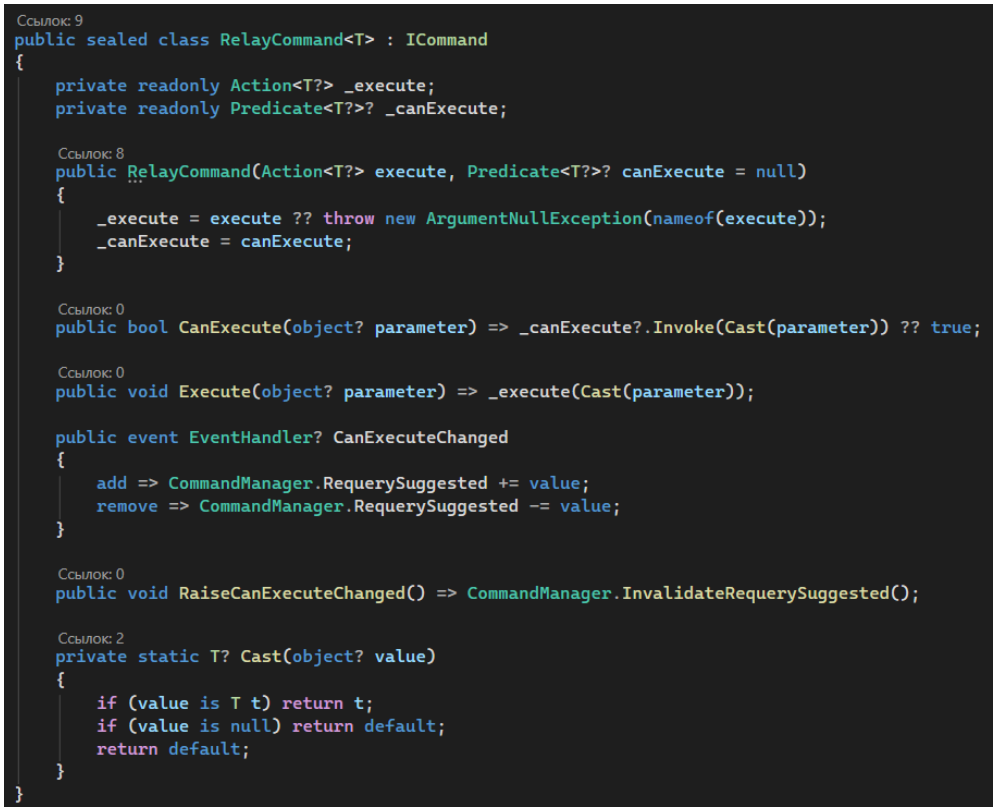
Рисунок 4.2 — ViewModel главной страницы (фрагмент)

Подключение ViewModel к View происходит через установку свойства DataContext. View-слой содержит только минимально необходимый code-behind — например, прокидывание содержимого PasswordBox в VM, так как PasswordBox не поддерживает прямой Binding из соображений безопасности. Пример реализации XAML-кода для страницы приведён в приложении Б.

4.3 Классы для реализации паттерна Command

Как уже было отмечено, паттерн MVVM вплотную взаимодействует с командами, реализованными через паттерн Command. В проекте используются два класса: RelayCommand и RelayCommand<T>.

RelayCommand — это команда без параметра. Она хранит делегаты Action (для Execute) и Func<bool> (для CanExecute) и пробрасывает CanExecuteChanged в CommandManager, чтобы WPF автоматически перезапрашивал состояние кнопок. RelayCommand<T> — это типизированная команда с параметром типа T, принимающая делегаты Action<T?> и Predicate<T?>.



```
Ссылок: 9
public sealed class RelayCommand<T> : ICommand
{
    private readonly Action<T?> _execute;
    private readonly Predicate<T?>? _canExecute;

    Ссылок: 8
    public RelayCommand(Action<T?> execute, Predicate<T?>? canExecute = null)
    {
        _execute = execute ?? throw new ArgumentNullException(nameof(execute));
        _canExecute = canExecute;
    }

    Ссылок: 0
    public bool CanExecute(object? parameter) => _canExecute?.Invoke(Cast(parameter)) ?? true;

    Ссылок: 0
    public void Execute(object? parameter) => _execute(Cast(parameter));

    public event EventHandler? CanExecuteChanged
    {
        add => CommandManager.RequerySuggested += value;
        remove => CommandManager.RequerySuggested -= value;
    }

    Ссылок: 0
    public void RaiseCanExecuteChanged() => CommandManager.InvalidateRequerySuggested();

    Ссылок: 2
    private static T? Cast(object? value)
    {
        if (value is T t) return t;
        if (value is null) return default;
        return default;
    }
}
```

Рисунок 4.3 — Реализация класса RelayCommand

Оба класса полноценно реализуют интерфейс ICommand и дополнительно помечены модификатором sealed, что полностью предотвращает возможность нежелательного наследования. Специальный метод RaiseCanExecuteChanged позволяет в нужный момент явно уведомить систему о необходимости повторно выполнить проверку текущей доступности конкретной команды.

Данные команды инициализируются непосредственно в конструкторе ViewModel как свойства, доступные только для чтения, и затем привязываются к визуальным элементам управления через стандартное свойство Command в разметке XAML. На рисунке 4.4 наглядно показан

пример создания и инициализации команды ToggleShowAllCommand в классе MainPageViewModel.

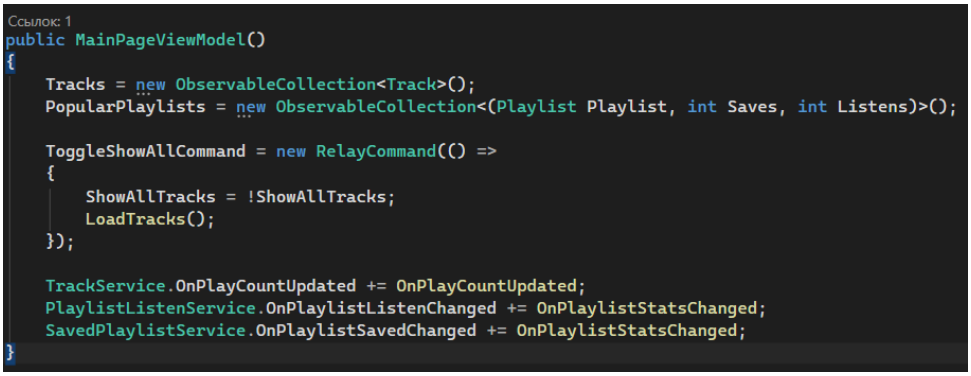


Рисунок 4.4 — Пример создания команды в ViewModel

Команды можно связывать как для стандартных событий (нажатие кнопки), так и для других событий через EventTrigger. Пример листинга ViewModel с командами приведён в приложении В.

4.4 Классы для реализации взаимодействия с базой данных

Для взаимодействия с базой данных PostgreSQL используется Entity Framework Core с провайдером Npgsql. Реализованы следующие компоненты: классы-сущности, контекст данных и сервисные классы.

Классы-сущности (User, Track, Playlist, Album, Statistic, Favorite и другие) описывают структуру таблиц базы данных. Свойства классов соответствуют столбцам таблиц, а аннотации ([Key], [Required], [MaxLength], [ForeignKey]) задают ограничения. Навигационные свойства описывают связи между сущностями. На рисунке 4.5 приведён пример сущности Track.

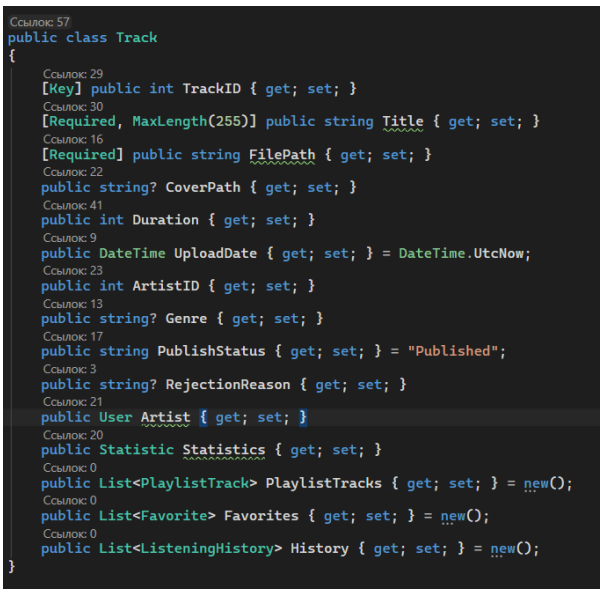


Рисунок 4.5 — Сущность Track

Класс AppDbContext наследуется от базового класса DbContext и содержит в себе свойства DbSet для абсолютно каждой таблицы данного проекта. В переопределенном методе OnConfiguring программно задаётся актуальная строка подключения к целевой базе данных PostgreSQL. В методе же OnModelCreating максимально детально настраиваются первичные ключи, уникальные индексы и все архитектурные связи между существующими таблицами. Полный листинг исходного кода класса AppDbContext целиком приведён в соответствующем приложении Г.

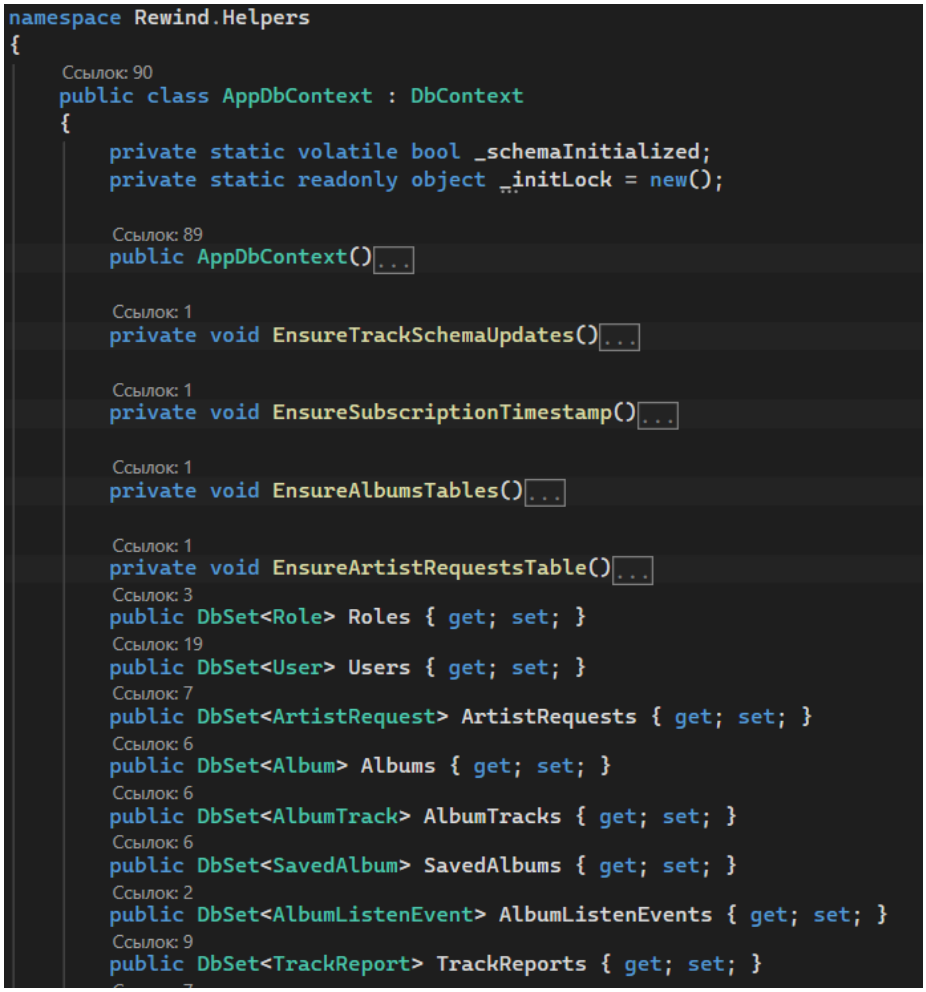


Рисунок 4.6 — Класс AppDbContext (фрагмент)

Вместо классического паттерна Repository в данном программном проекте целенаправленно применяются статические сервисные классы: TrackService, UserService, PlaylistService, FavoriteService, AlbumService, ArtistRequestService, TrackReportService и другие. Каждый такой специализированный класс содержит в себе набор методов для работы с соответствующей сущностью. Каждый метод обязательно создаёт новый экземпляр AppDbContext внутри блока using, что гарантирует корректное закрытие соединения и полное освобождение ресурсов сразу после

выполнения операции. Фрагмент кода класса TrackService детально изображён на рисунке 4.7.

```
namespace Rewind.Helpers
{
    Ссылок: 37
    public static class TrackService
    {
        public static event Action<int, int>? OnPlayCountUpdated;
        public static event Action<int, string, string>? NewTrackUploaded;
        Ссылок: 1
        public static void NotifyNewTrackUploaded(int artistId, string artistName, string trackTitle)
            => NewTrackUploaded?.Invoke(artistId, artistName, trackTitle);

        Ссылок: 0
        public static List<Track> GetAllTracks()
        {
            using var db = new AppDbContext();
            return db.Tracks.Include(t => t.Artist).Include(t => t.Statistics).ToList();
        }

        Ссылок: 9
        public static Track? GetTrackById(int id)
        {
            using var db = new AppDbContext();
            return db.Tracks.Include(t => t.Artist).Include(t => t.Statistics)
                            .FirstOrDefault(t => t.TrackID == id);
        }

        Ссылок: 0
        public static List<Track> GetTracksByArtist(int artistId)
        {
            using var db = new AppDbContext();
            return db.Tracks.Include(t => t.Statistics)
                            .Where(t => t.ArtistID == artistId).ToList();
        }
```

Рисунок 4.7 — Фрагмент класса TrackService

Статический класс Session хранит данные текущего пользователя (UserId, UserName, Email, UserRole, AvatarPath и другие) в памяти на протяжении всей сессии. При запуске приложения данные загружаются из базы методом LoadFromDatabase, а при выходе — сохраняются методом FlushToDatabase, вызываемым из App.OnExit.

4.5 Методы для работы с файлами

В разработанном программном средстве реализована полноценная работа с файлами: загрузка аудиофайлов, изображений обложек и аватаров пользователей, а также их последующее сохранение в локальные папки приложения. На текущий момент поддерживаются следующие форматы аудио: MP3, WAV. Для изображений же предусмотрена поддержка форматов JPEG, PNG, WebP.

Статический класс FileStorage обеспечивает централизованную работу со всеми файлами. Метод CopyTrackAudio копирует выбранный аудиофайл в папку MusicLibrary, предварительно выполняя очистку имени файла от любых недопустимых символов. Для точного определения продолжительности трека успешно используется внешняя библиотека TagLibSharp. На рисунке 4.8

приведён подробный фрагмент метода загрузки трека из класса ArtistStudioViewModel.

```
Ссылок: 1
private void UploadTrack()
{
    if (string.IsNullOrWhiteSpace(NewTrackTitle) || string.IsNullOrEmpty(NewTrackAudioPath))
    { _dialog.Error("Введите название и выберите аудиофайл!"); return; }

    try
    {
        string fileName = FileStorage.CopyTrackAudio(NewTrackAudioPath, NewTrackTitle);
        string destPath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "MusicLibrary", fileName);
        int duration;
        using (var f = TagLib.File.Create(destPath))
            duration = (int)f.Properties.Duration.TotalSeconds;

        string? coverPath = !string.IsNullOrWhiteSpace(NewTrackCoverPath)
            ? FileStorage.CopyTrackCover(NewTrackCoverPath) : null;

        var track = new Track
        {
            Title = NewTrackTitle,
            FilePath = fileName,
            CoverPath = coverPath,
            Duration = duration,
            UploadDate = DateTime.UtcNow,
            ArtistID = Session.UserId,
            Genre = NewTrackGenre,
            PublishStatus = "Pending"
        };
        TrackService.AddTrack(track);
        _dialog.Info("Трек отправлен на проверку администратору.", "Заявка отправлена");

        NewTrackTitle = ""; NewTrackAudioPath = null; NewTrackCoverPath = null;
        UploadFinished?.Invoke();
    }
    catch (Exception ex) { _dialog.Error(ex.Message); }
}
```

Рисунок 4.8 — Фрагмент метода загрузки аудиофайла

Методы CopyAvatar, CopyTrackCover, CopyPlaylistCover, CopyAlbumCover копируют соответствующие изображения в специализированные папки (Images/Avatars, Images/TrackCovers и другие). Метод ResolvePath преобразует сохранённый в базе относительный путь в абсолютный путь файловой системы. Листинг класса FileStorage приведён в приложении Д.

Для диалогов выбора файлов используется класс DialogService, реализующий интерфейс IDialogService. Метод PickAudio() открывает системный диалог выбора файла с фильтром по аудиоформатам (*.mp3, *.wav), метод PickImage() — с фильтром по изображениям.

4.2 Методы для регистрации и авторизации пользователей

При запуске приложения пользователь попадает в окно регистрации/авторизации. Вся бизнес-логика регистрации и входа реализована в классе RegistrationViewModel. View-слой лишь прокидывает содержимое PasswordBox через свойства VM.

При регистрации выполняются следующие проверки:

— корректность формата email (допускаются адреса на gmail.com, yandex.ru, yandex.com, ya.ru, mail.ru);

— соответствие никнейма требованиям (3-20 символов, только латинские буквы, цифры и знак подчёркивания);
— надёжность пароля (не менее 8 символов, хотя бы одна буква и одна цифра, без пробелов);
— совпадение пароля и подтверждения пароля;
— уникальность email и никнейма в системе.

Для хранения паролей используется алгоритм PBKDF2 с SHA-256 и случайной солью (100 000 итераций). Реализация содержится в классе PasswordHelper. Метод HashPassword генерирует случайную соль, вычисляет хеш и возвращает строку формата «соль:хеш» в кодировке Base64. На рисунке 4.9 приводится фрагмент метода HashPassword.

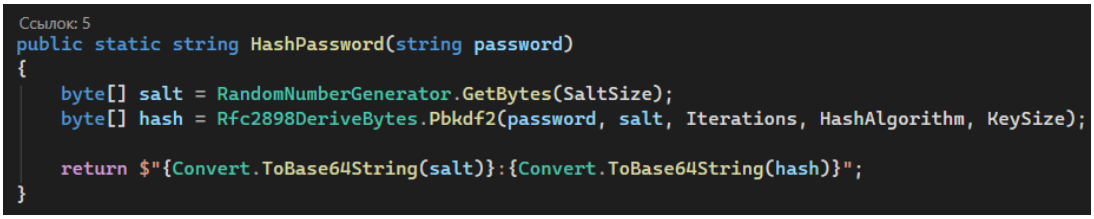


Рисунок 4.9 — Метод хеширования пароля

При регистрации исполнителя пользователь не получает сразу доступ к системе: его данные помещаются в таблицу ArtistRequests со статусом Pending. Администратор рассматривает заявку в панели администратора на вкладке «Заявки исполнителей». При одобрении создаётся полноценный аккаунт исполнителя.

При авторизации хранимый хеш пароля сравнивается с хешем введённого пароля методом VerifyPassword с использованием CryptographicOperations.FixedTimeEquals для защиты от атак по времени. Дополнительно проверяется соответствие выбранной роли роли пользователя в базе данных. В зависимости от роли пользователь перенаправляется в главное окно (MainWindow) или в панель администратора (AdminPanel). Листинг класса RegistrationViewModel приведён в приложении Е.

5 Тестирование, проверка работоспособности и анализ полученных результатов

5.1 Тестировании регистрации и авторизации

Начнём тестирование с окна авторизации. Первым делом попробуем ввести неверные учётные данные — несуществующую почту или неправильный пароль. Результат демонстрируется на рисунке 5.1.

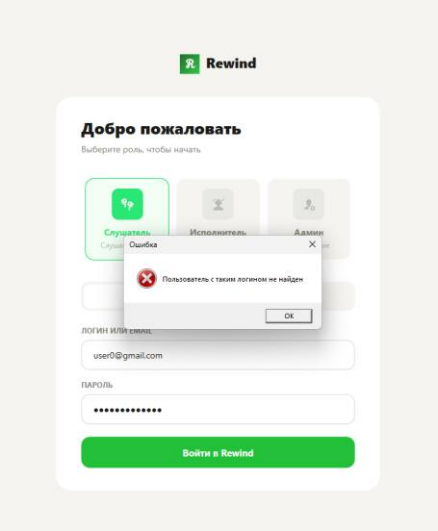


Рисунок 5.1 – Ошибка авторизации

Следующим этапом попробуем оставить все поля для заполнения пустыми и нажать кнопку входа. В результате будет выведено соответствующее сообщение о необходимости заполнения всех полей. Демонстрируется это на рисунке 5.2.

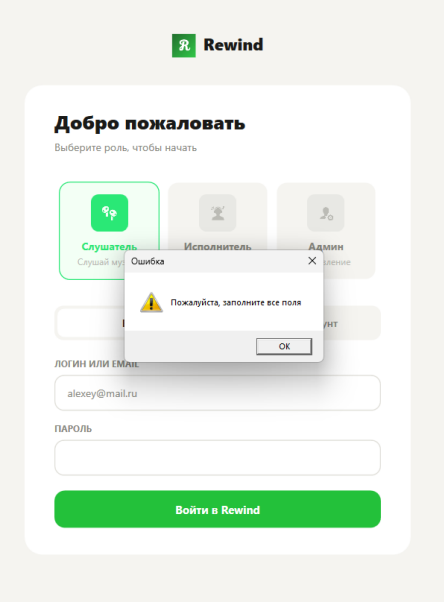


Рисунок 5.2 – Ошибка при незаполненных полях

Перейдём к тестированию регистрации. Попробуем зарегистрировать нового пользователя, указав почту, которая уже используется в системе. На рисунке 5.3 демонстрируется результат, в котором выводится соответствующее сообщение о том, что пользователь с такой почтой уже существует.

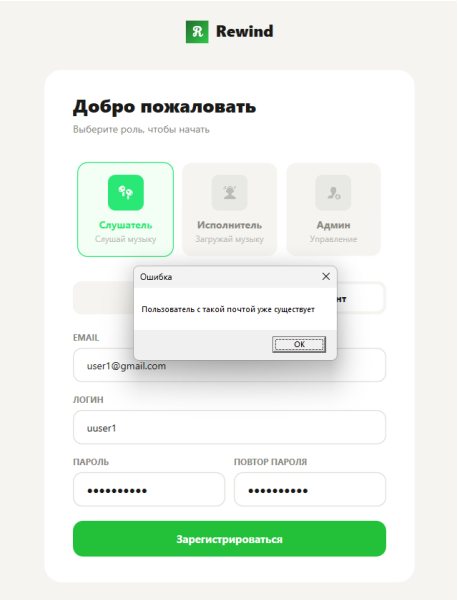


Рисунок 5.3 – Ошибка регистрации

Далее укажем неверный формат адреса электронной почты, без символа «@». На рисунке 5.4 показано окно, которое будет выведено пользователю.

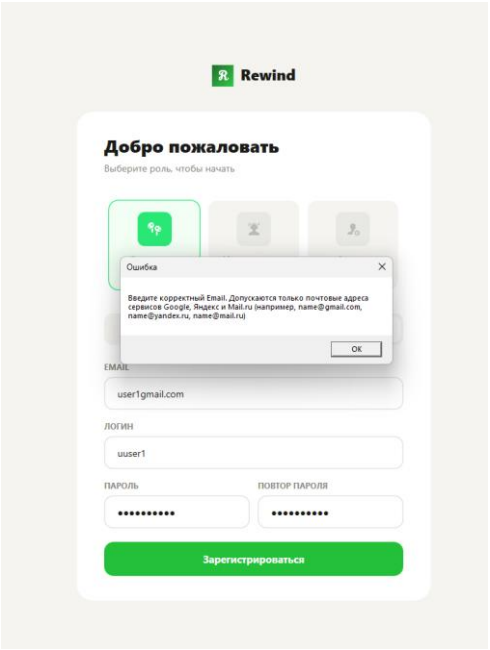


Рисунок 5.4 – Ошибка ввода электронной почты

Попробуем также ввести слишком короткий пароль. Результат данной операции показан на рисунке 5.5.

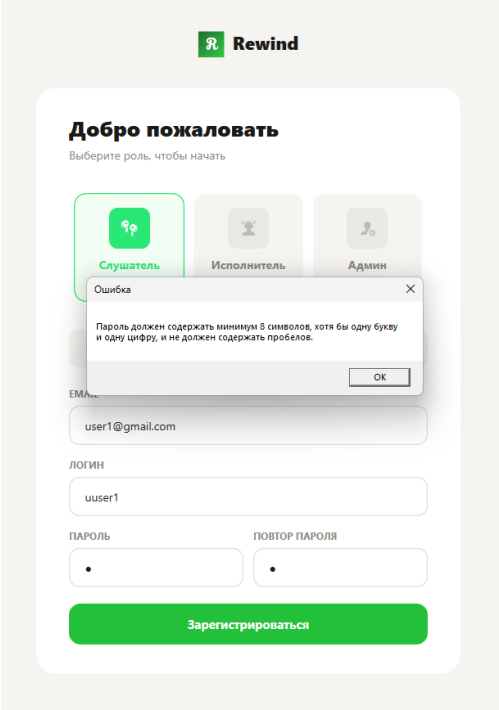


Рисунок 5.5 – Ошибка при вводе слабого пароля

Дополнительно проверим случай, когда пароль и его подтверждение не совпадают. Соответствующее сообщение отображено на рисунке 5.6.

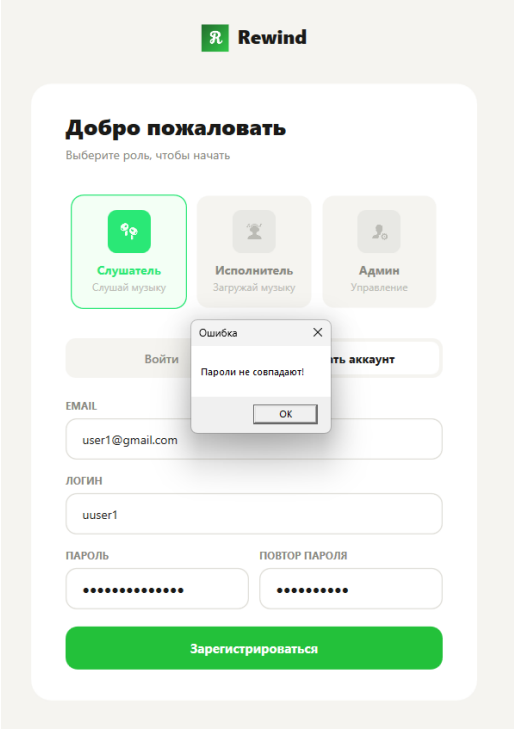


Рисунок 5.6 – Ошибка при несовпадении паролей

Таким образом, можно сделать вывод, что окна авторизации и регистрации корректно обрабатывают все основные случаи возникновения ошибок.

5.2 Тестирование работы с музыкальным плеером

Основой функционала приложения является воспроизведение музыкальных треков. Проверим корректность работы плеера. При выборе трека из любого списка (главная страница, плейлист, альбом, избранное) он должен начать проигрываться, а в нижней части окна отобразится панель плеера с названием трека, исполнителем и обложкой. Результат запуска воспроизведения демонстрируется на рисунке 5.7.

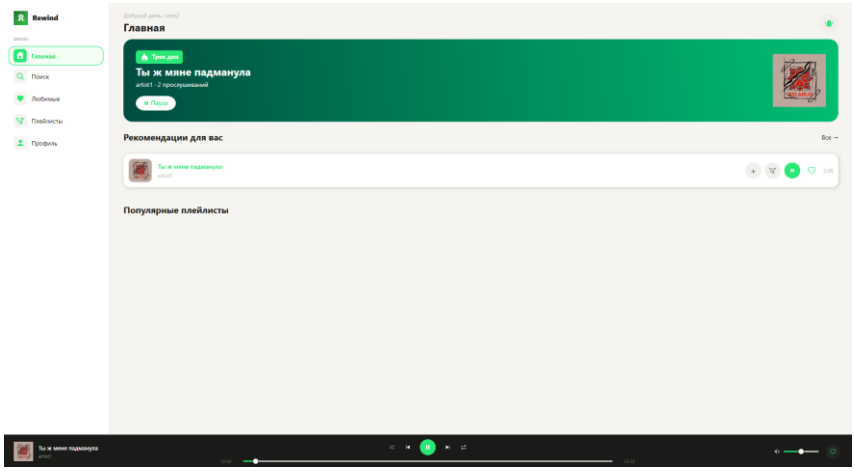


Рисунок 5.7 – Панель плеера при воспроизведении трека

Далее протестируем кнопки управления воспроизведением: пауза, продолжение, переход к следующему и предыдущему треку. Все кнопки корректно выполняют свои функции, полоса прогресса отображает текущую позицию в треке. Результат показан на рисунке 5.8.

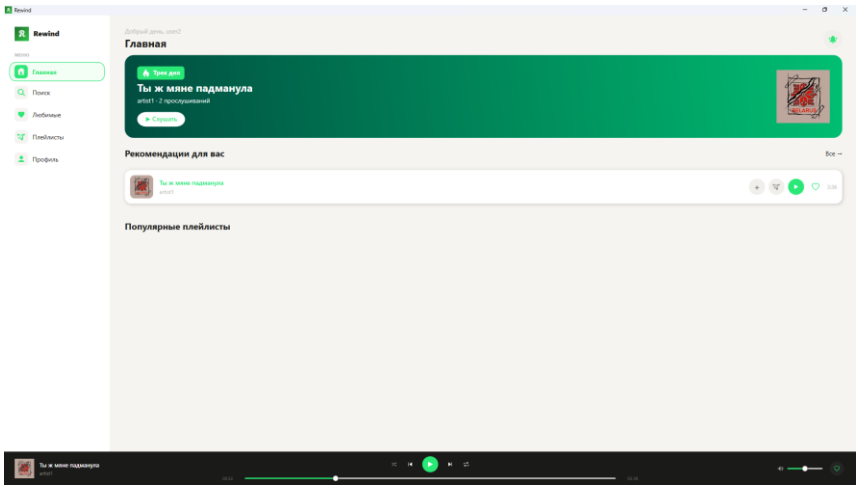


Рисунок 5.8 – Управление воспроизведением

Проверим возможность перемотки трека путём перетаскивания ползунка. При этом время воспроизведения обновляется, а звук переходит к выбранной позиции. Результат представлен на рисунке 5.9.

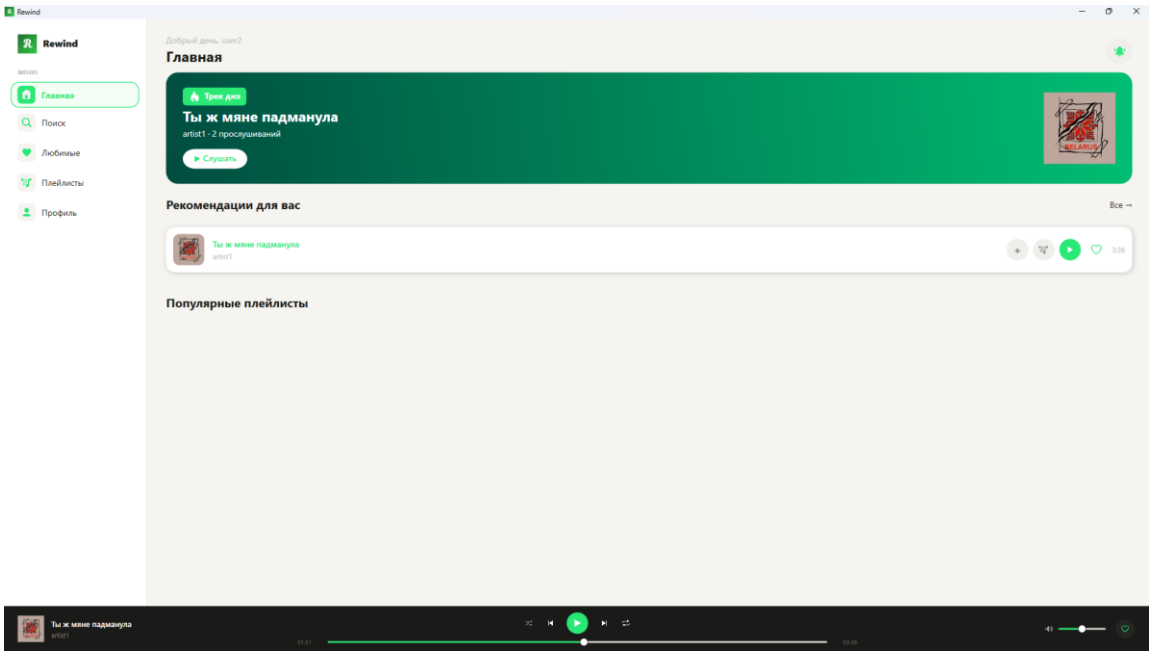


Рисунок 5.9 – Перемотка трека

Также протестируем изменение громкости. Результат данной операции показан на рисунке 5.10.

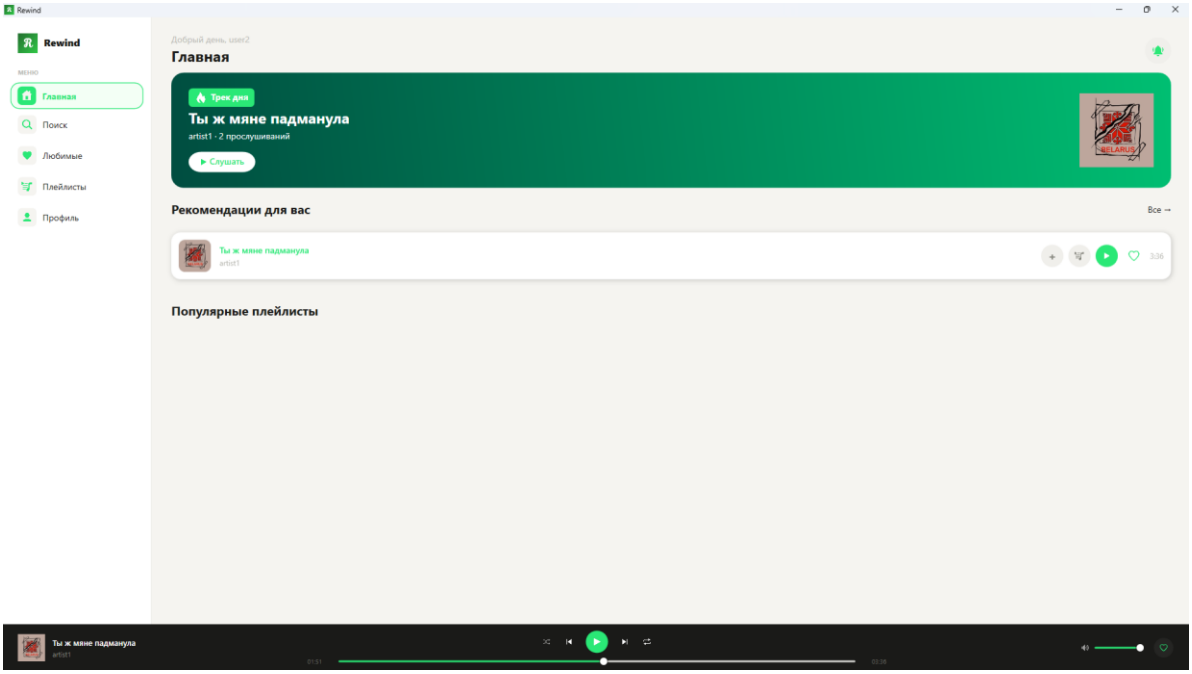


Рисунок 5.10 – Регулировка громкости

Отдельно проверим переход на страницу «Сейчас играет», на которой отображается увеличенная обложка трека, его название, исполнитель и дополнительные элементы управления. Демонстрация приведена на рисунке 5.11.

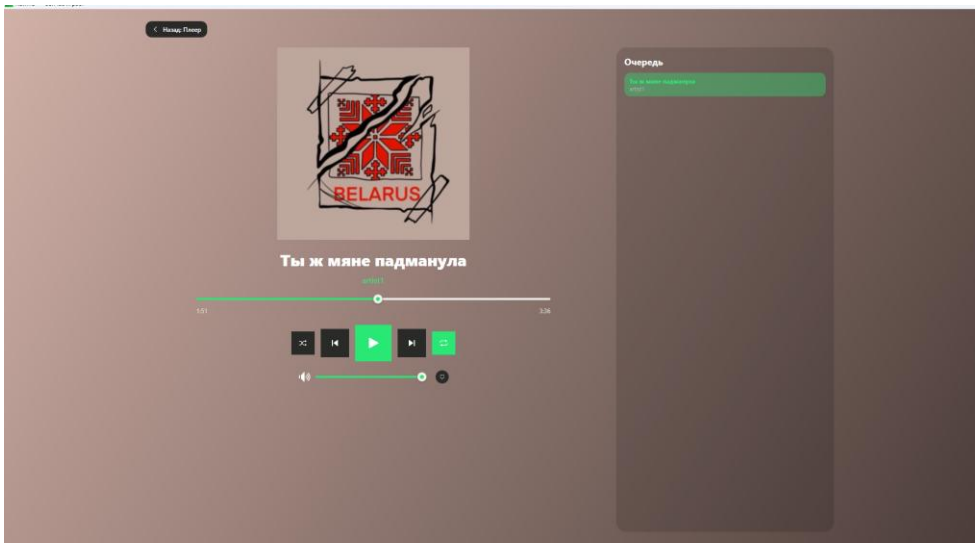


Рисунок 5.11 – Страница «Сейчас играет»

5.3 Тестирование создания и редактирования плейлистов

Одной из ключевых возможностей приложения является создание пользовательских плейлистов. Первым делом попробуем создать плейлист, оставив поле названия пустым. Результат данной операции отображён на рисунке 5.12.

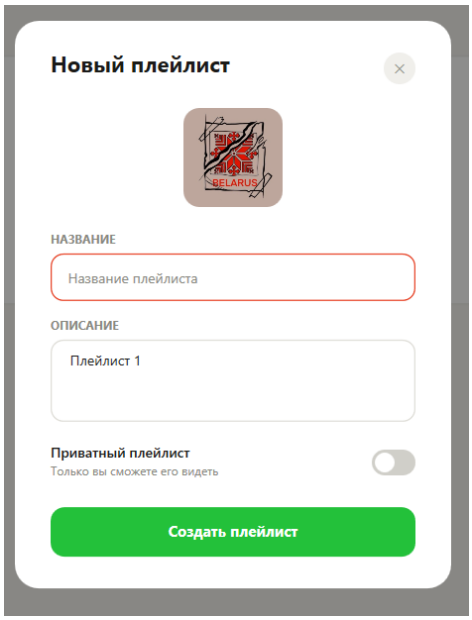


Рисунок 5.12 – Ошибка при создании плейлиста без названия

Далее создадим плейлист с корректным названием и описанием. В результате плейлист появится на странице «Плейлисты» пользователя. Демонстрируется это на рисунке 5.13.



Рисунок 5.13 – Успешное создание плейлиста

Проверим возможность добавления треков в плейлист через контекстное меню трека. После добавления трек должен появиться в плейлисте. Результат показан на рисунке 5.14.

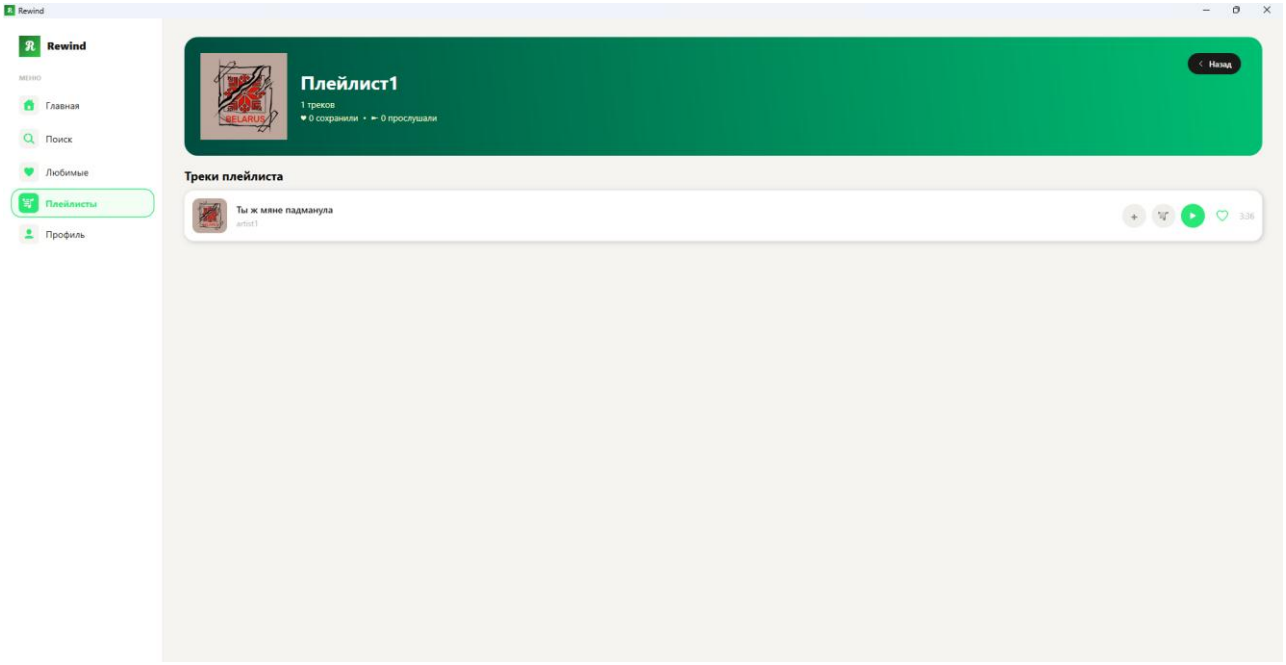


Рисунок 5.14 – Добавление трека в плейлист

Попробуем добавить один и тот же трек в плейлист дважды. В этом случае будет выведено сообщение о том, что трек уже присутствует в плейлисте. Данный результат отображён на рисунке 5.15.

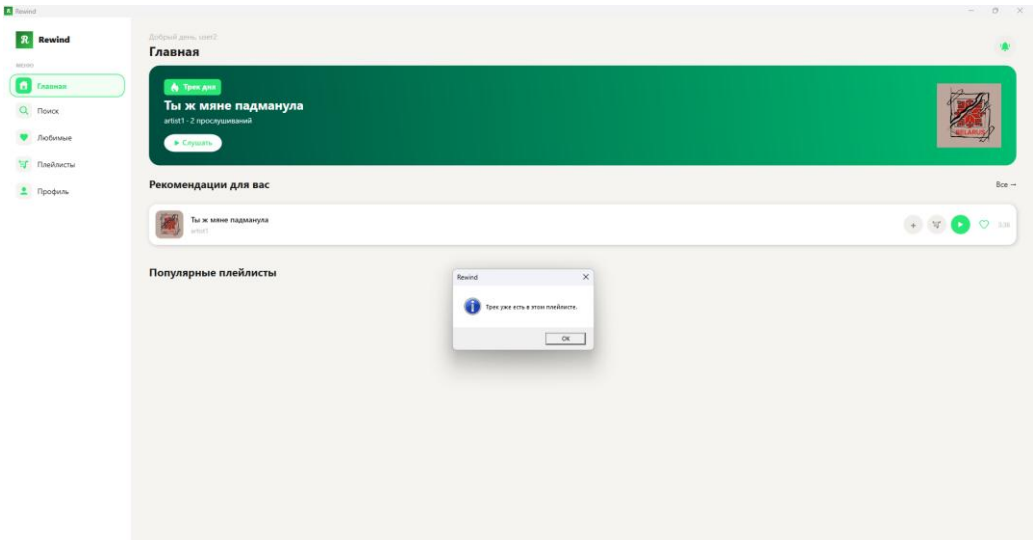


Рисунок 5.15 – Ошибка при повторном добавлении трека

Далее проверим возможность удаления трека из плейлиста и удаления самого плейлиста. Перед удалением плейлиста выводится окно подтверждения, чтобы избежать случайной потери данных. Результат представлен на рисунке 5.16.

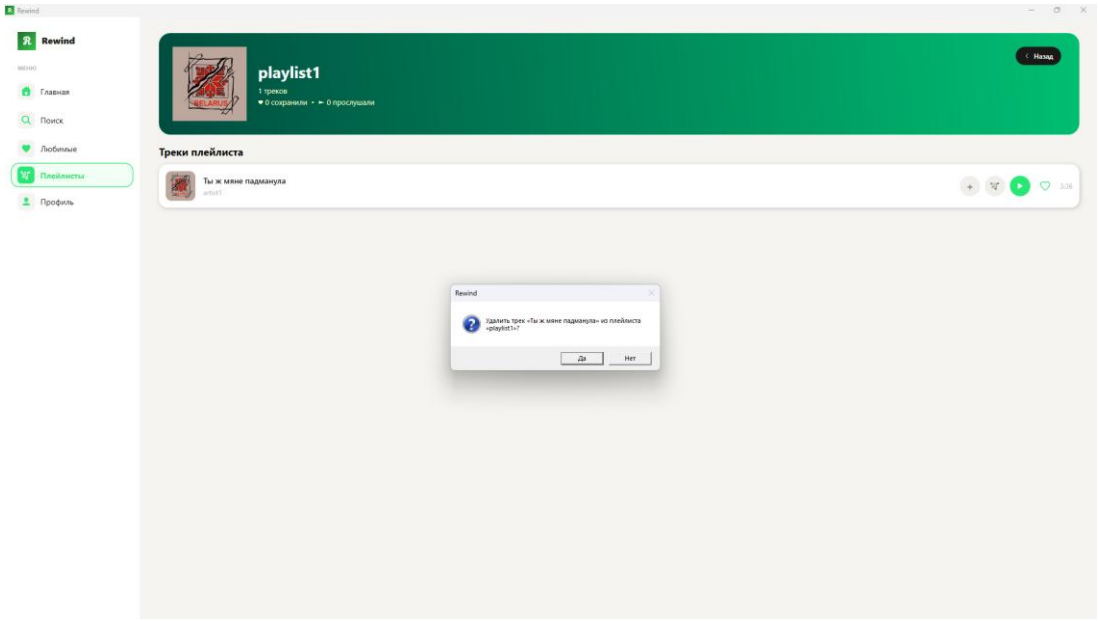


Рисунок 5.16 – Подтверждение удаления плейлиста

5.4 Тестирование поиска

Проверим поведение системы при вводе запроса, по которому ничего не найдено. В этом случае будет выведено сообщение об отсутствии результатов. Результат на рисунке 5.17.



Рисунок 5.17 – Отсутствие результатов поиска

Отдельно проверим поиск при пустом поле ввода и при вводе только пробелов. В обоих случаях поиск не выполняется, а пользователю предлагается ввести запрос. Результат показан на рисунке 5.18.

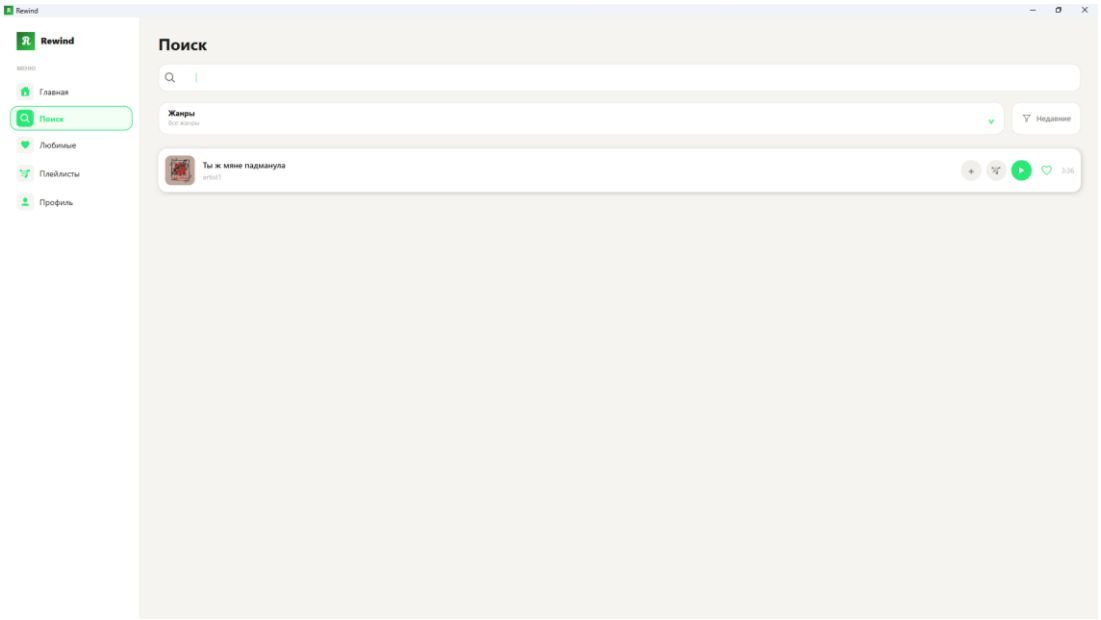


Рисунок 5.18 – Поведение при пустом запросе

5.5 Тестирование страницы избранного

Проверим возможность добавления трека в «Любимые треки» по нажатию на соответствующую иконку. После добавления трек появляется на странице «Любимые треки». Результат демонстрируется на рисунке 5.19.

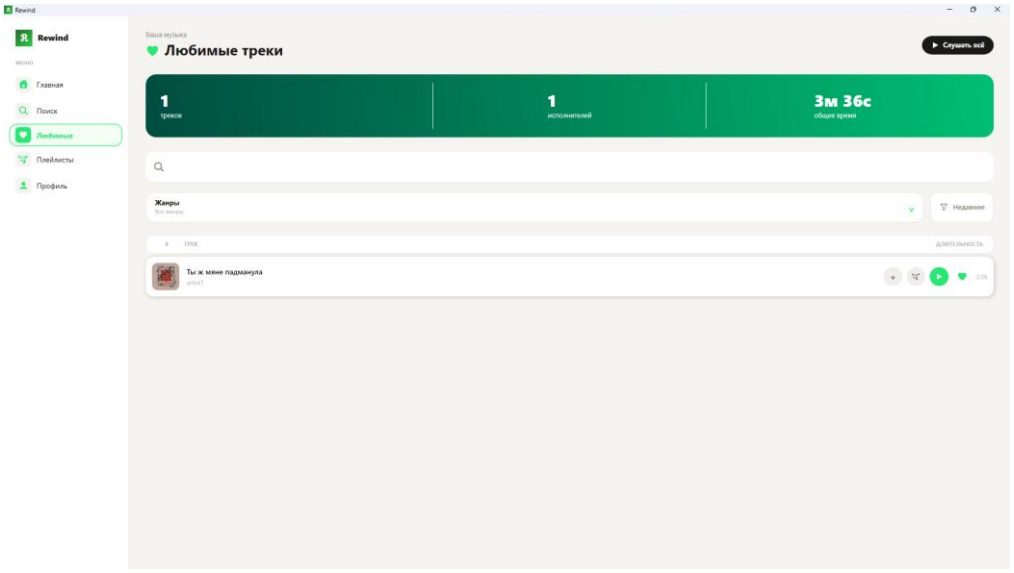


Рисунок 5.19 – Добавление трека в избранное

Повторное нажатие на иконку удаляет трек из избранного. Треку соответствующим образом изменяется внешний вид кнопки. Результат отображён на рисунке 5.20.

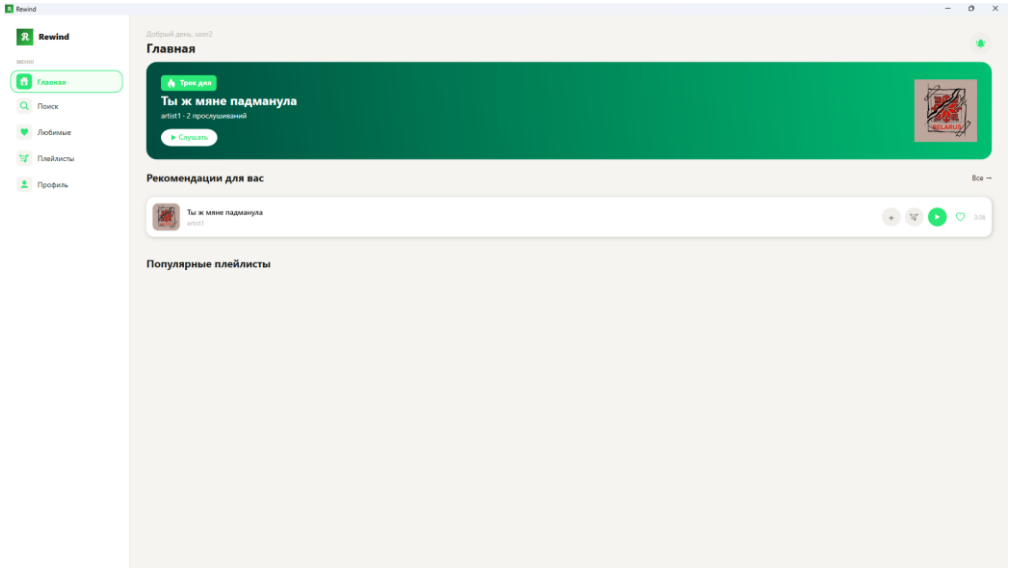


Рисунок 5.20 – Удаление трека из избранного

5.6 Тестирование студии исполнителя

Пользователь, имеющий статус исполнителя, получает доступ к странице «Студия», где он может загружать свои треки, создавать альбомы и просматривать аналитику прослушиваний. Первым делом попробуем загрузить трек, не заполнив обязательные поля. В результате будет выведено сообщение об ошибке, которое демонстрируется на рисунке 5.21.

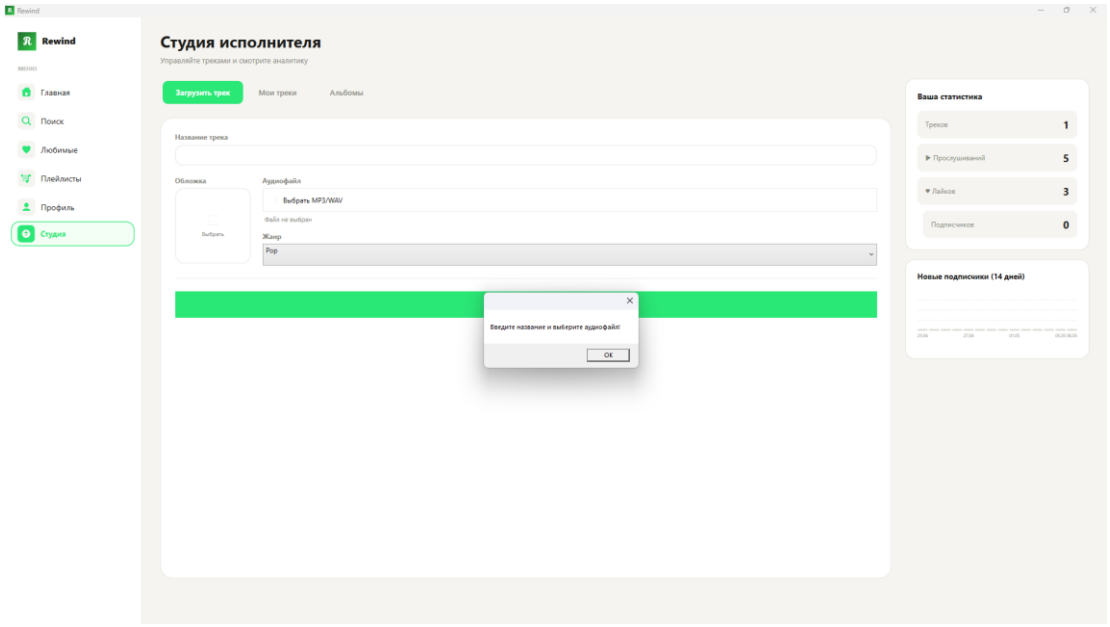


Рисунок 5.21 – Ошибка при загрузке трека без обязательных полей

Далее корректно заполним все поля и загрузим трек. После успешной загрузки трек отправляется на модерацию администратору, о чём пользователю выводится соответствующее сообщение. Результат на рисунке 5.22.

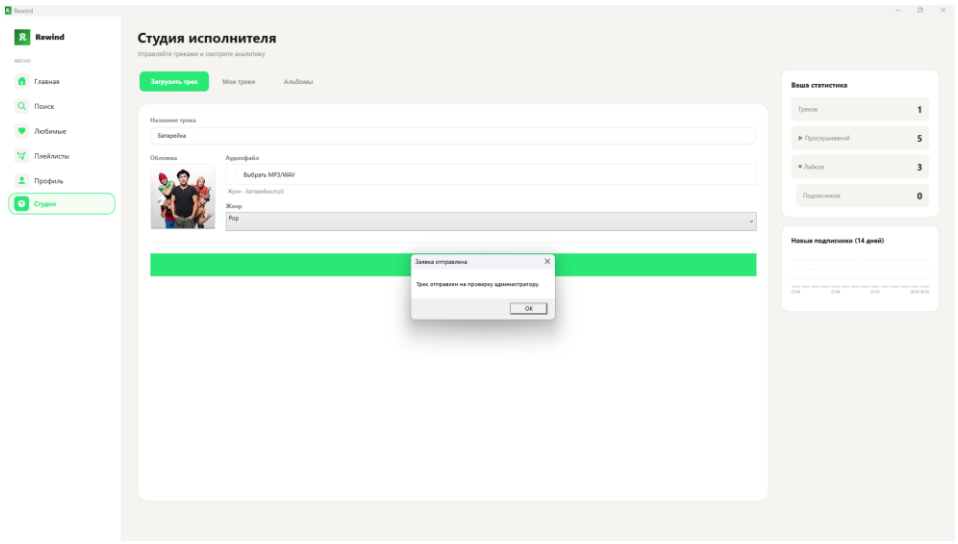


Рисунок 5.22 – Успешная отправка трека на модерацию

Также проверим раздел аналитики, в котором исполнитель может видеть количество прослушиваний своих треков и изменения этого показателя во времени. Результат представлен на рисунке 5.23.

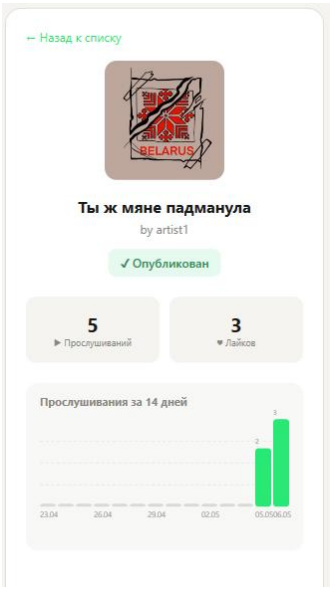


Рисунок 5.23 – Аналитика прослушиваний треков исполнителя

5.7 Тестирование жалоб на треки

Далее корректно заполним форму и отправим жалобу. После успешной отправки пользователю выводится подтверждающее сообщение, а жалоба появляется в панели администратора. Результат показан на рисунке 5.24.

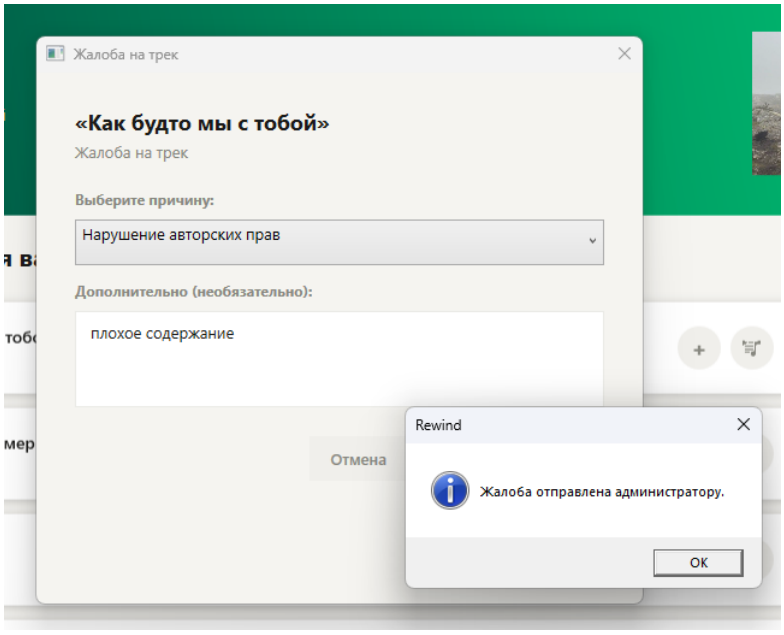


Рисунок 5.24 – Успешная подача жалобы на трек

5.8 Тестирование профиля пользователя

Проверим функционал страницы профиля. Пользователь может изменить имя, аватар и прочие данные. Попробуем сохранить пустое имя — будет выведено сообщение об ошибке. Результат отображён на рисунке 5.25.

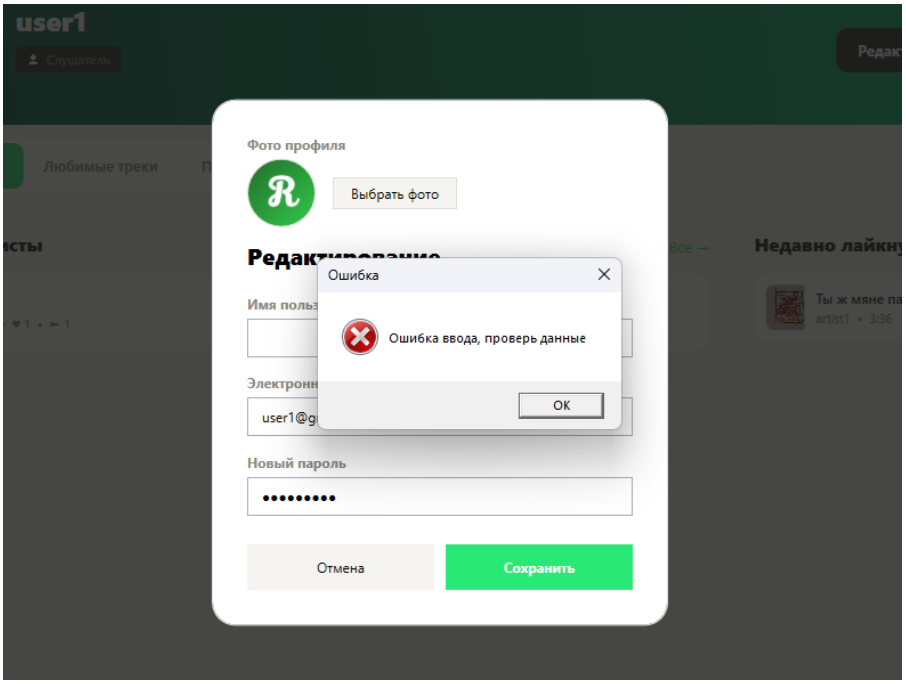


Рисунок 5.25 – Ошибка при сохранении пустого имени

Корректно изменим данные профиля и сохраним изменения. Результат успешного обновления показан на рисунке 5.26.

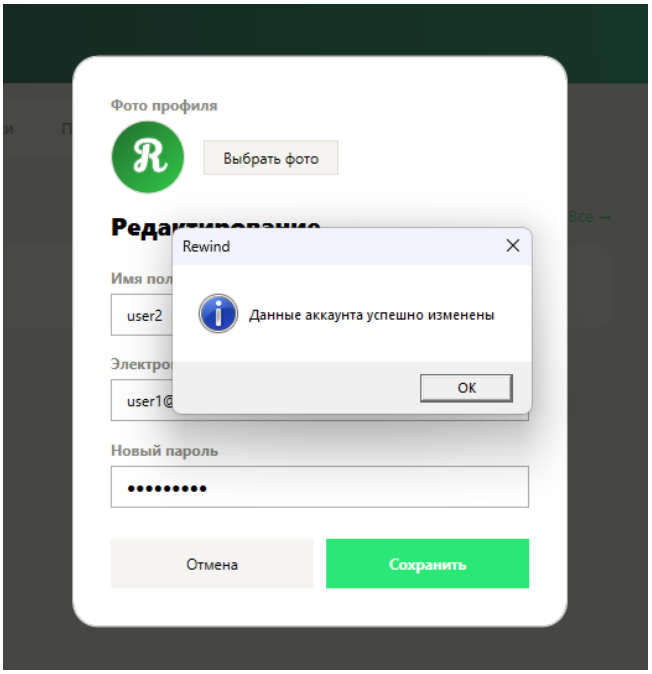


Рисунок 5.26 – Успешное обновление профиля

Также проверим переключение темы приложения (светлая/тёмная). Все элементы интерфейса корректно перекрашиваются согласно выбранной теме. Результат представлен на рисунке 5.27.

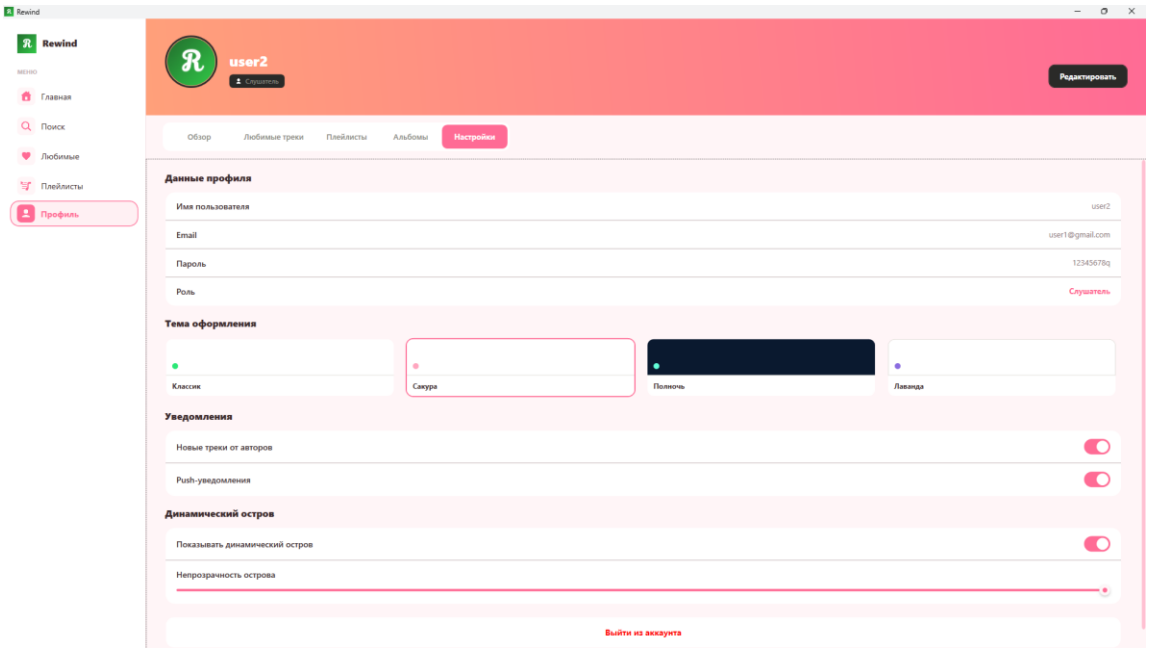


Рисунок 5.27 – Переключение темы оформления

5.9 Тестирование панели администратора

Администратор имеет доступ к расширенным возможностям: управление пользователями, модерация треков, обработка заявок на статус исполнителя и рассмотрение жалоб. Проверим вкладку «Пользователи». Администратор может заблокировать пользователя. Результат показан на рисунке 5.28.

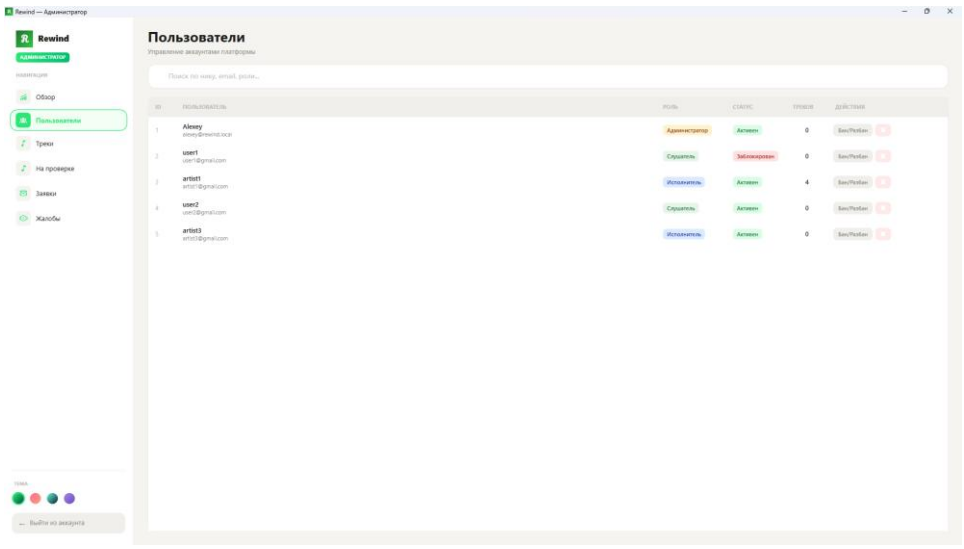


Рисунок 5.28 – Блокировка пользователя

Проверим вкладку «Заявки исполнителей». Администратор может одобрить или отклонить заявку. Результат отображён на рисунке 5.29.

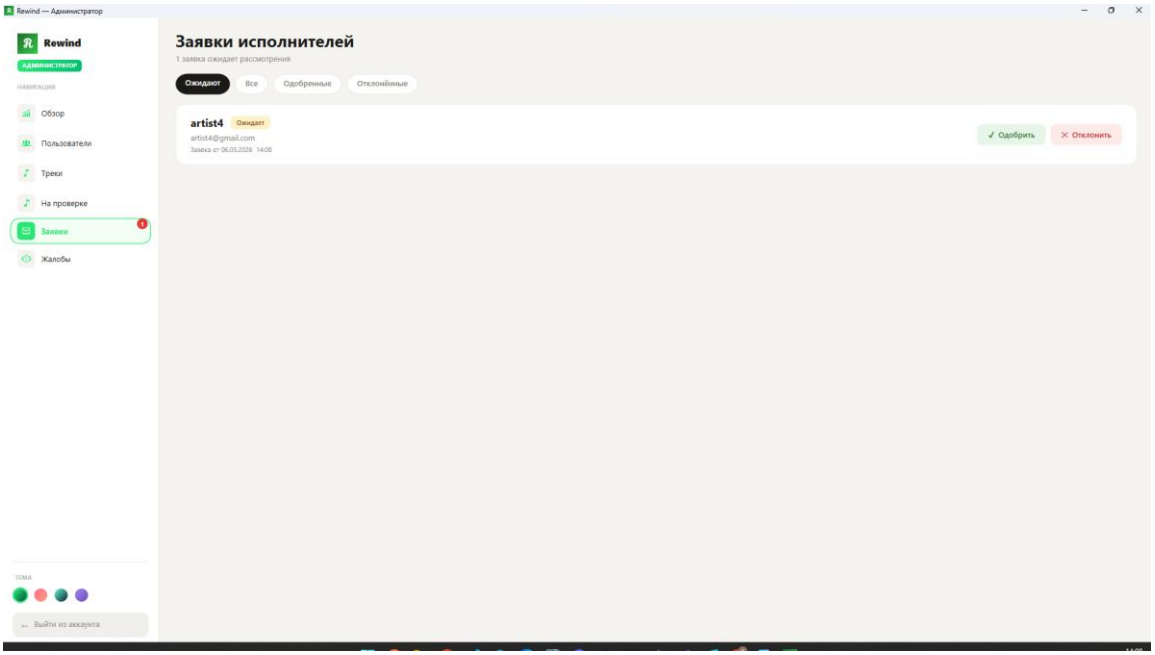


Рисунок 5.29 – Возможность принять или отклонить заявку

Далее проверим вкладку модерации треков. Администратор прослушивает загруженный исполнителем трек и принимает решение о его одобрении или отклонении. После одобрения трек становится доступен всем пользователям, а исполнителю приходит уведомление. Результат на рисунке 5.30.

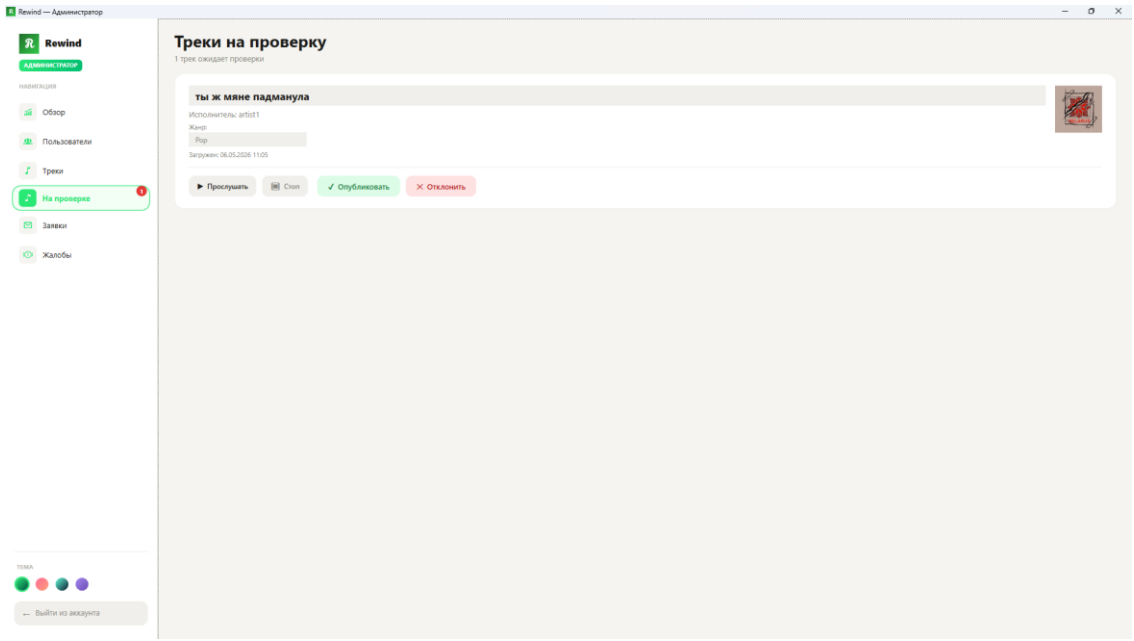


Рисунок 5.30 – Одобрение трека администратором

Проверим вкладку «Жалобы на треки». Администратор может просмотреть текст жалобы, прослушать трек и принять решение — удалить трек либо отклонить жалобу. Результат представлен на рисунке 5.31.

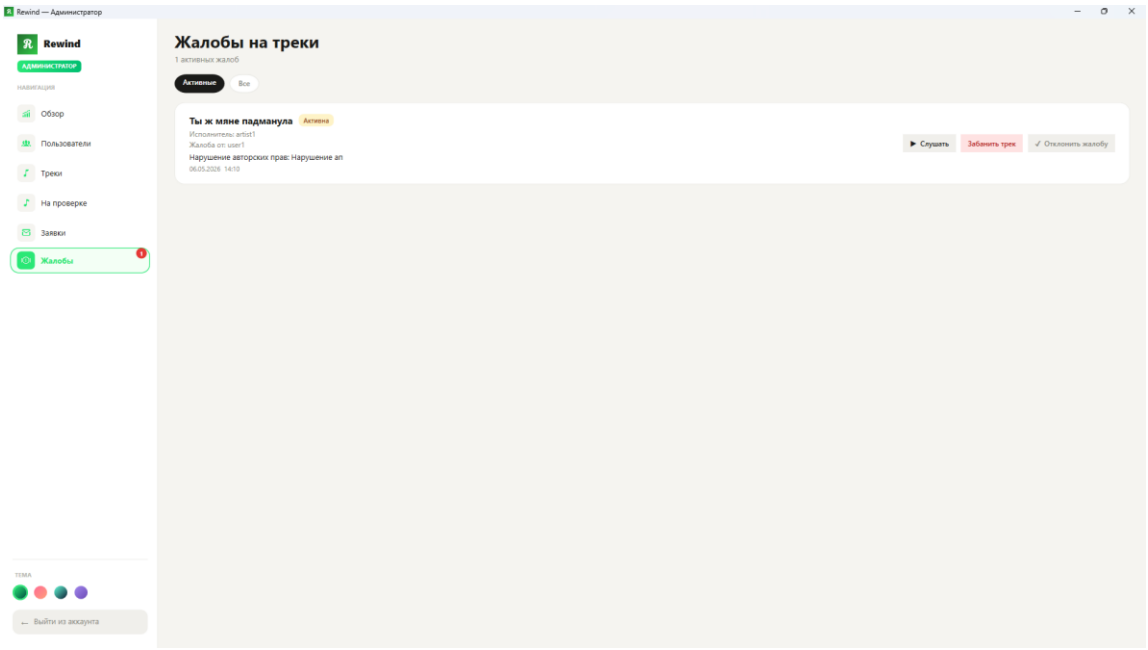


Рисунок 5.31 – Рассмотрение жалобы администратором

Также протестируем раздел общей статистики («Обзор»), в котором отображаются ключевые показатели приложения: количество пользователей, треков, плейлистов, активность прослушиваний. Результат показан на рисунке 5.32.

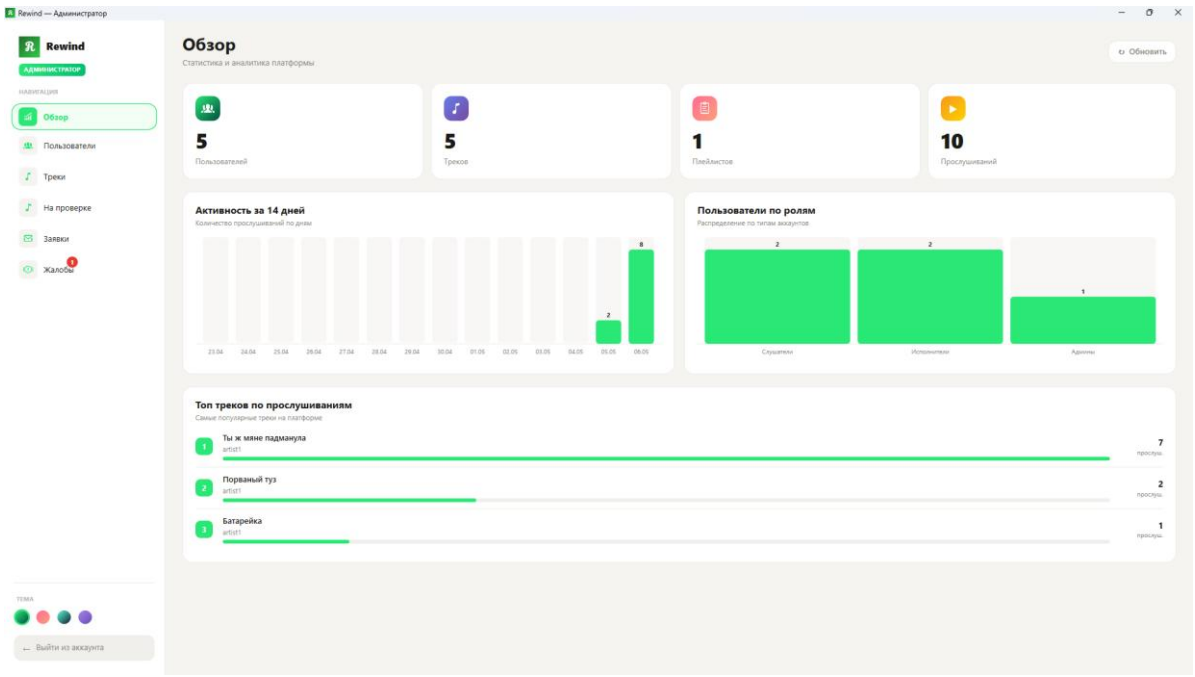


Рисунок 5.32 – Общая статистика в панели администратора

5.10 Тестирование альбомов и страниц исполнителей

Проверим открытие страницы альбома. На странице отображаются обложка альбома, название, исполнитель, список треков и кнопки воспроизведения. Результат показан на рисунке 5.33.

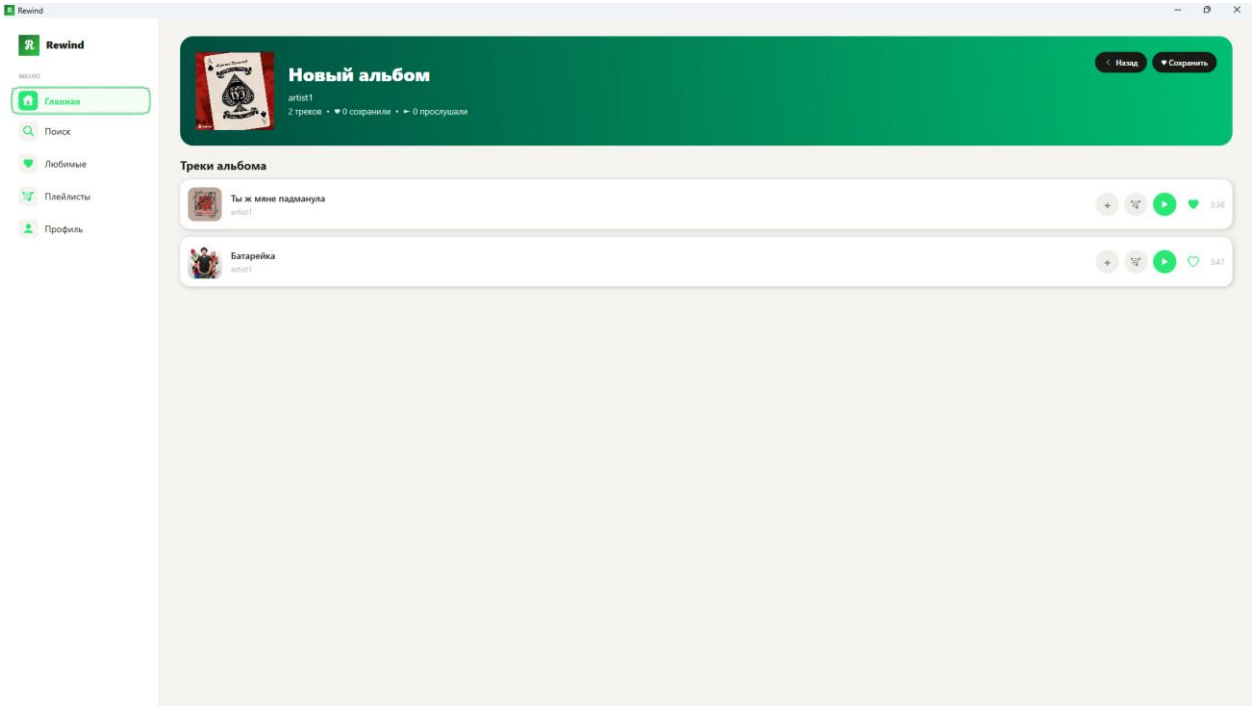


Рисунок 5.33 – Страница альбома

Проверим переход на страницу исполнителя. Отображаются его имя, аватар, список альбомов и популярных треков. Результат представлен на рисунке 5.34.

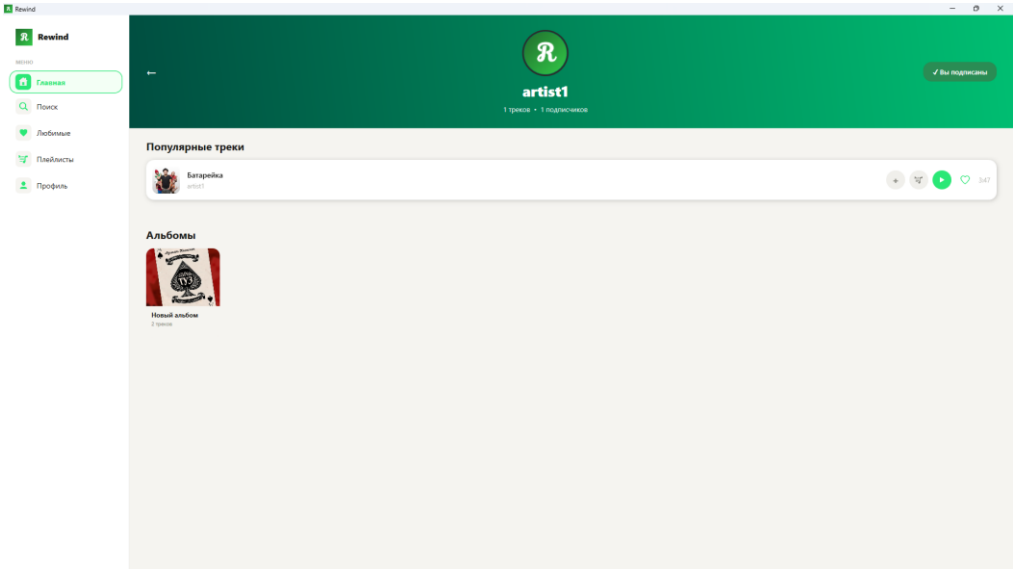


Рисунок 5.34 – Страница исполнителя

Также протестируем кнопку подписки на исполнителя. После нажатия внешний вид кнопки меняется, а исполнитель появляется в списке подписок пользователя. Результат на рисунке 5.35.

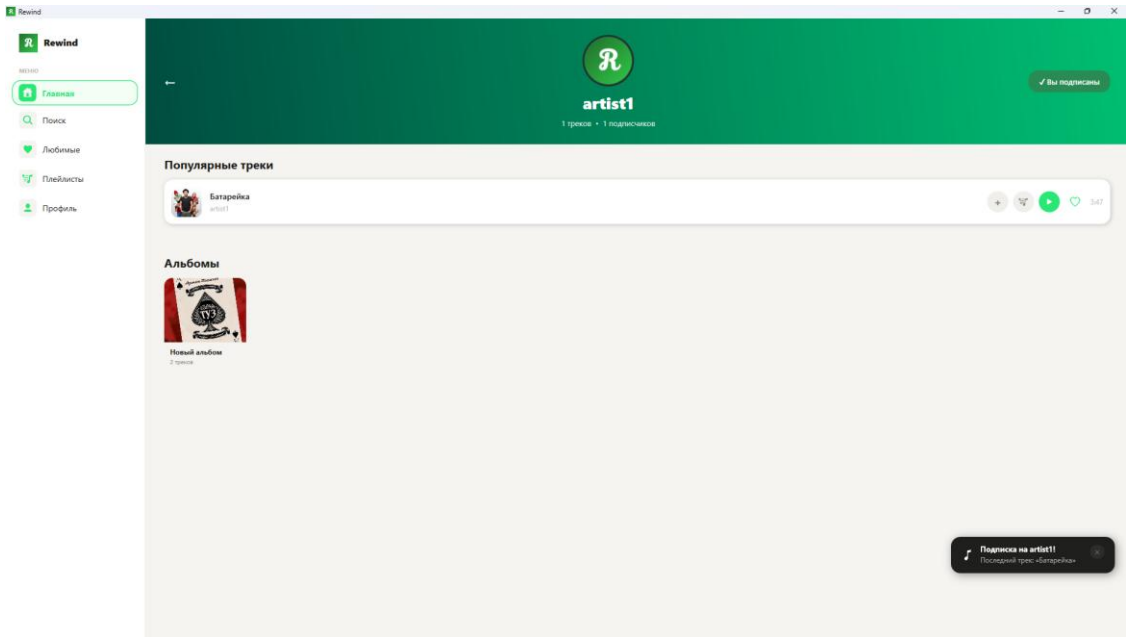


Рисунок 5.35 – Подписка на исполнителя

В результате проведённого тестирования можно сделать вывод, что все ключевые модули приложения Rewind функционируют корректно. Формы ввода данных обрабатывают основные случаи некорректного заполнения, выводя пользователю понятные сообщения об ошибках. Плеер стабильно воспроизводит треки и поддерживает весь необходимый функционал управления. Взаимодействие между пользовательской частью и панелью администратора работает согласованно: загруженные исполнителями треки проходят модерацию, жалобы обрабатываются, а статистика отображается в актуальном виде. Таким образом, приложение готово к использованию и устойчиво к типовым ошибкам пользователя.

6 Руководство по использованию

При запуске приложения «Rewind» открывается окно регистрации и авторизации (рисунок 6.1). Пользователю необходимо выбрать роль (Слушатель, Исполнитель или Администратор) и ввести данные для входа, либо зарегистрироваться, нажав соответствующую кнопку.

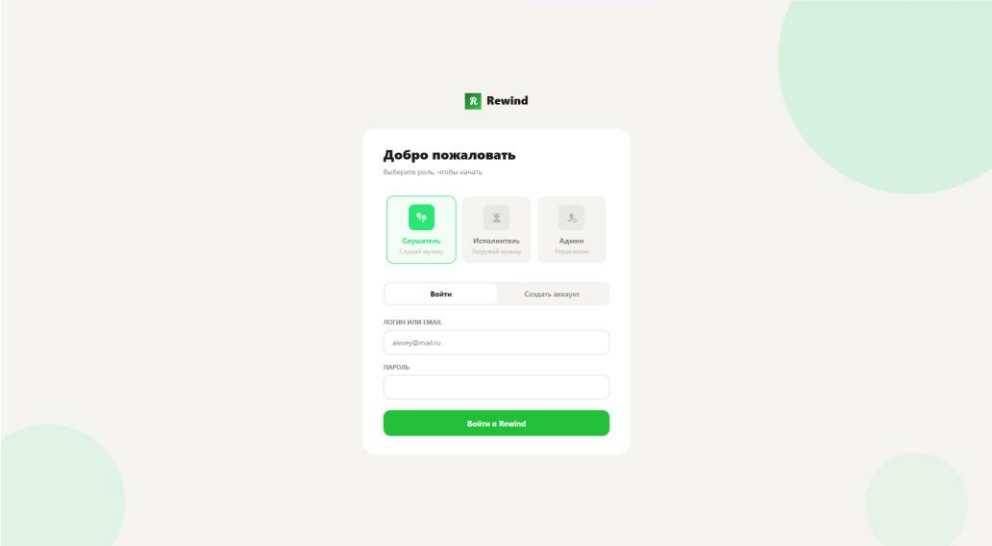


Рисунок 6.1 — Окно авторизации

При отсутствии учётной записи необходимо перейти на вкладку «Регистрация» и заполнить все поля формы: email, никнейм и пароль. После успешной регистрации пользователь автоматически входит в систему и переходит в главное окно приложения.

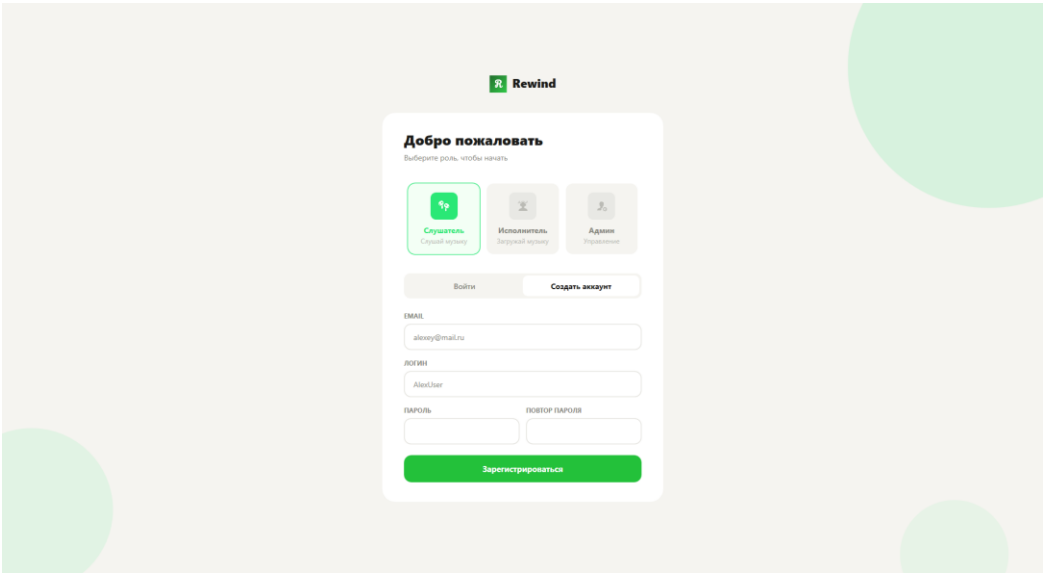


Рисунок 6.2 — Окно регистрации

После успешной авторизации слушатель или исполнитель попадает в главное окно приложения. Главное окно содержит боковую панель навигации и в нижней части — панель плеера. По умолчанию отображается главная страница с рекомендуемым треком дня и популярными плейлистами (рисунок 6.3).

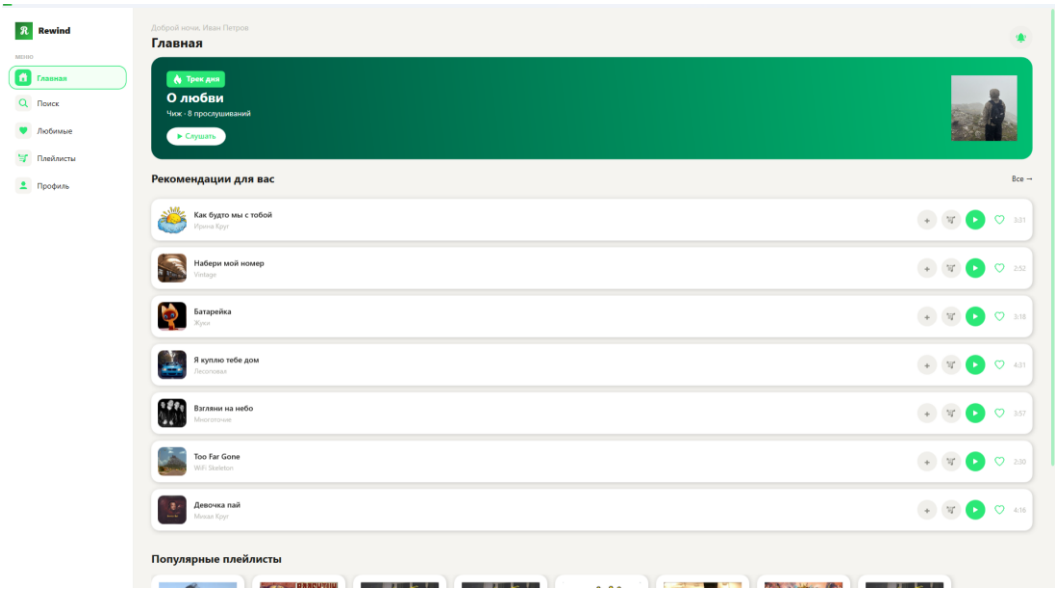


Рисунок 6.3 — Главная страница приложения

Для воспроизведения трека необходимо нажать на него левой кнопкой мыши. Трек начнёт воспроизводиться, а в нижней панели плеера отобразятся название трека, имя исполнителя и обложка. С помощью панели плеера можно управлять воспроизведением (рисунок 6.4):

— кнопки «Предыдущий» / «Следующий» — переход между треками в очереди;

— кнопка «Пауза/Воспроизведение» — приостановка и возобновление воспроизведения;

— ползунок прогресса — перемотка трека в любое место;

— ползунок громкости — регулировка уровня звука;

— кнопка «Повтор» — включение/отключение повторного воспроизведения очереди;

— кнопка «Перемешать» — случайный порядок треков в очереди.



Рисунок 6.4 — Панель плеера

Для поиска треков необходимо перейти на страницу «Поиск» (рисунок 6.5). На этой странице можно ввести поисковый запрос по названию трека или имени исполнителя, выбрать один или несколько жанров в выпадающем

списке, а также циклически переключать порядок сортировки результатов (по названию, исполнителю или длительности) нажатием кнопки сортировки.

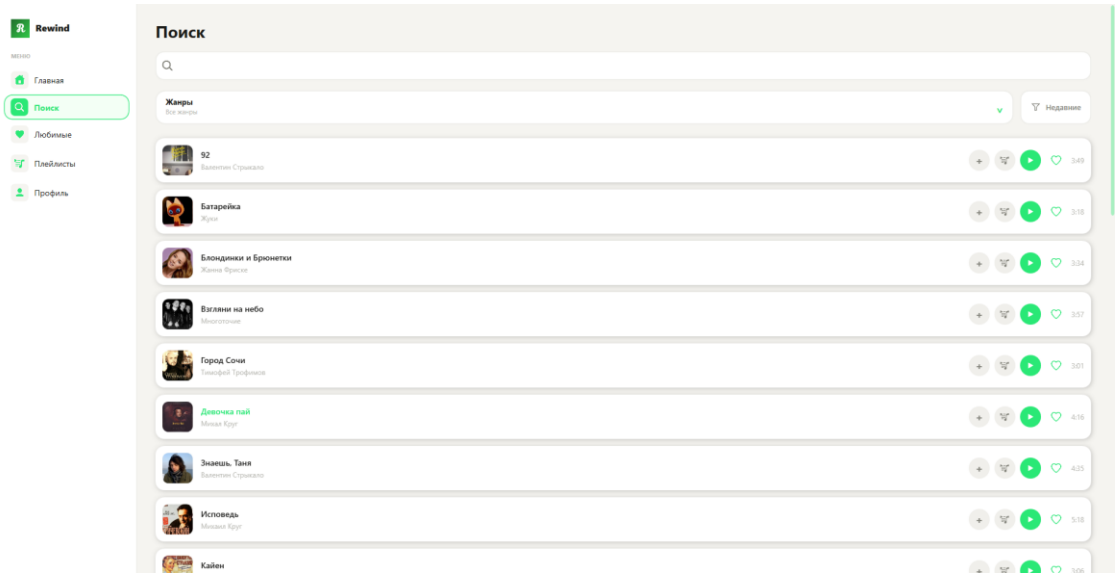


Рисунок 6.5 — Страница поиска

На странице «Плейлисты» (рисунок 6.6) пользователь может просматривать собственные и сохранённые плейлисты, создавать новые плейлисты, задавая их название, обложку и уровень приватности (публичный или приватный). Для добавления трека в плейлист необходимо нажать правой кнопкой мыши на трек в любом списке и выбрать нужный плейлист из контекстного меню.

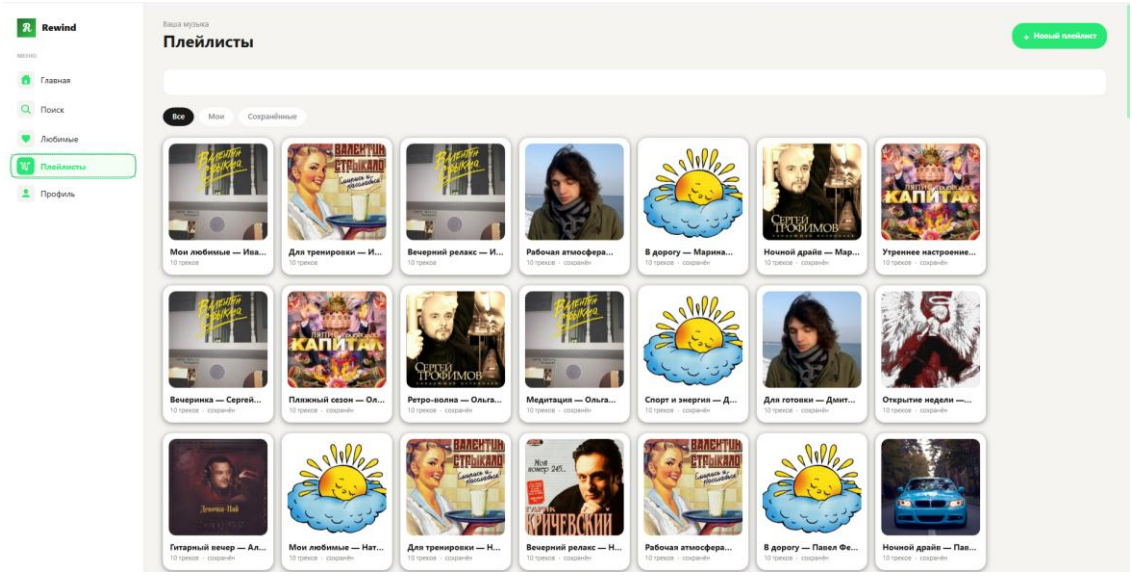


Рисунок 6.6 — Страница плейлистов

На странице «Избранное» (рисунок 6.7) отображаются все треки, которые пользователь добавил в избранное нажатием кнопки-лайка. Доступна

сортировка по дате добавления, названию, исполнителю и длительности, а также фильтрация по жанрам.

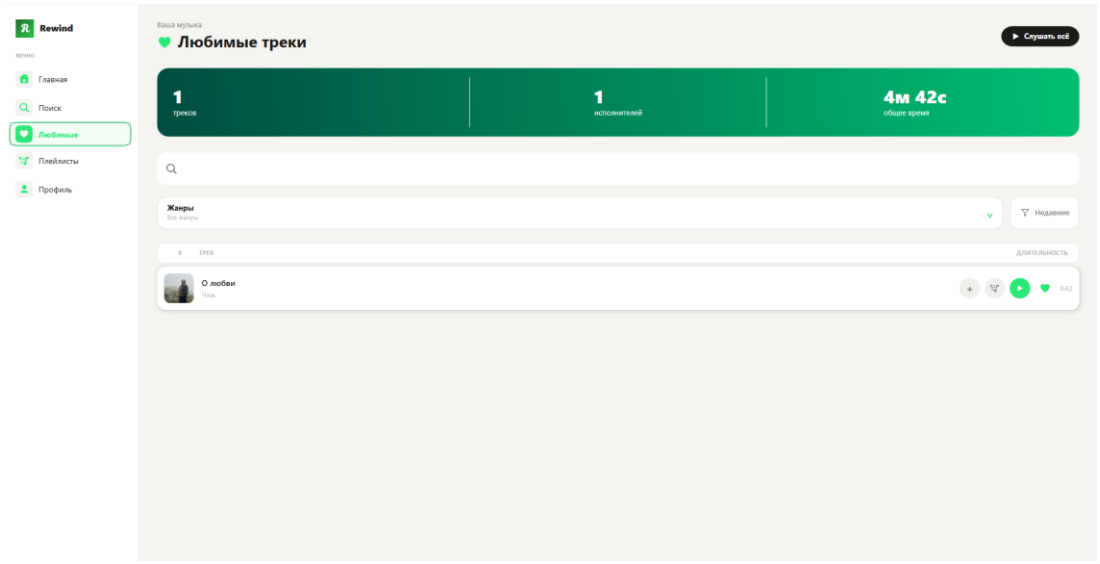


Рисунок 6.7 — Страница избранного

На странице «Профиль» (рисунок 6.8) пользователь может просматривать и редактировать данные аккаунта (никнейм, email, пароль, аватар), изменять тему оформления (Classic, Dark, Strawberry, Sea, Grass), просматривать собственные и сохранённые плейлисты, понравившиеся треки и сохранённые альбомы.

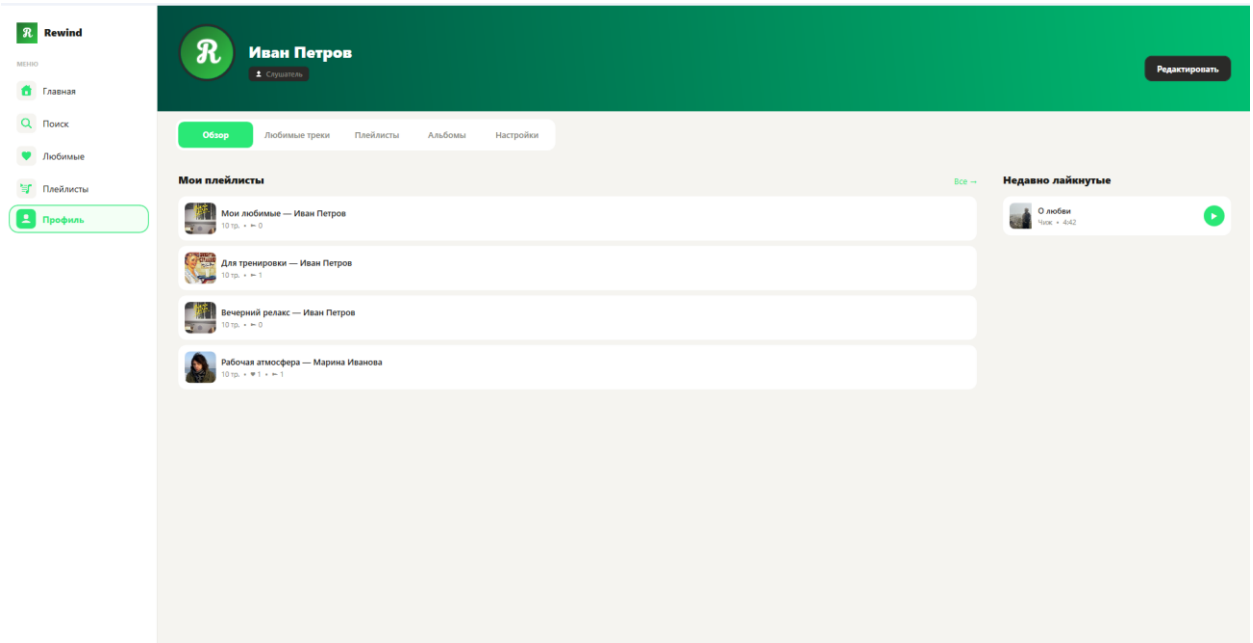


Рисунок 6.8 — Страница профиля

Для исполнителей доступна страница «Студия» (рисунок 6.9), на которой можно загрузить новый трек, указав название, жанр и прикрепив

аудиофайл и обложку. Также здесь можно создать или редактировать альбом, добавив в него треки из собственной библиотеки, и просматривать список загруженных треков со статусами публикации.

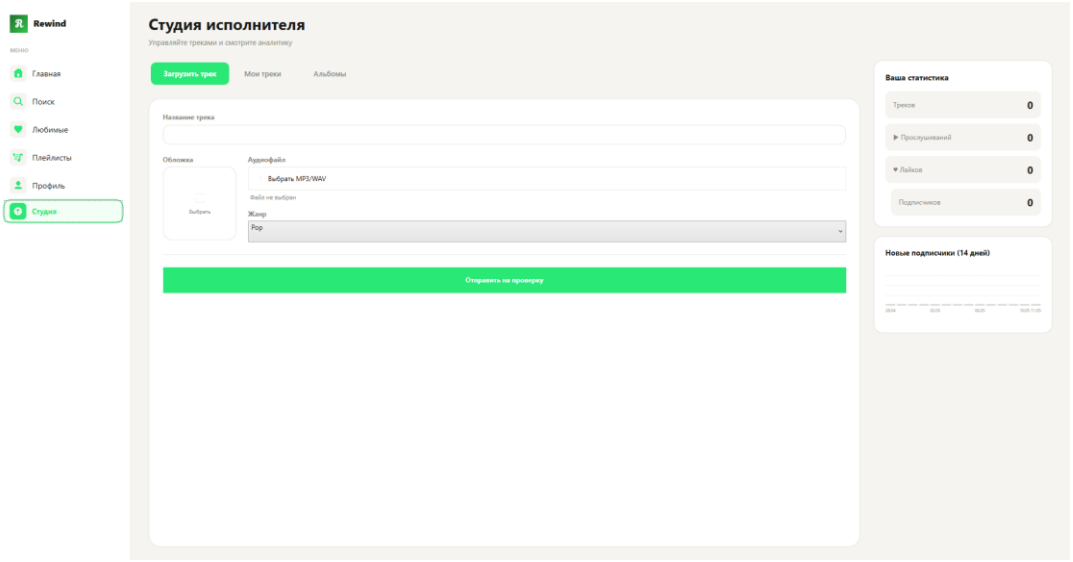


Рисунок 6.9 — Студия исполнителя

При нажатии на обложку в панели плеера открывается окно «Сейчас играет» (NowPlaying), отображающее увеличенную обложку трека, его название и исполнителя с дополнительными элементами управления. Также предусмотрен «Остров» — компактный плавающий мини-плеер (IslandWindow), отображающийся поверх всех окон при переключении на другое приложение, что позволяет управлять воспроизведением без возврата в основное окно.

Администратор после авторизации попадает в административную панель. На вкладке «Заявки исполнителей» отображается список заявок на регистрацию. Администратор может одобрить заявку, создав аккаунт исполнителя, или отклонить её. На вкладке «Загрузки треков» отображаются треки, ожидающие публикации: администратор может прослушать их и принять решение об их публикации или отклонении с указанием причины. На вкладке «Жалобы» отображаются жалобы пользователей, которые также рассматриваются администратором.

ЗАКЛЮЧЕНИЕ

В рамках данного курсового проекта было разработано программное средство «Rewind» — настольное приложение для воспроизведения и организации музыкальных коллекций. В ходе проектирования и реализации были достигнуты следующие результаты.

Проведён анализ существующих аналогов (Spotify, AIMP и других), на основе которого сформированы требования к разрабатываемому программному средству. Выявлены ключевые функциональные и нефункциональные требования, определены три роли пользователей (слушатель, исполнитель, администратор) с соответствующими функциональными возможностями.

Спроектирована и реализована реляционная база данных из 18 таблиц в СУБД PostgreSQL 17, развёрнутой в Docker-контейнере. Взаимодействие с базой данных осуществляется посредством Entity Framework Core с подходом Code-First и провайдером Npgsql.

Приложение построено на архитектурном паттерне MVVM, что обеспечивает чёткое разделение слоёв представления и бизнес-логики. Для слабой связанности компонентов применён паттерн Service Locator. Команды реализованы с помощью паттерна Command (классы RelayCommand и RelayCommand<T>).

Реализован полноценный музыкальный плеер с поддержкой очереди воспроизведения, перемешивания, повтора, перемотки и регулировки громкости. Разработаны страницы: главная, поиск, плейлисты, избранное, профиль, студия исполнителя, а также административная панель с вкладками управления пользователями, модерации треков и рассмотрения жалоб. Реализованы компактный плавающий мини-плеер (IslandWindow) и полноэкранный режим (NowPlaying).

Реализована безопасная система регистрации с хешированием паролей по алгоритму PBKDF2/SHA-256 с солью, ролевая модель авторизации и гибкая система тем оформления. Для загрузки и хранения медиафайлов разработан вспомогательный класс FileStorage, а для определения метаданных аудиофайлов задействована библиотека TagLibSharp.

В процессе разработки применены следующие технологии: C#/.NET 10, WPF, Entity Framework Core, PostgreSQL, Docker, TagLibSharp, Figma.

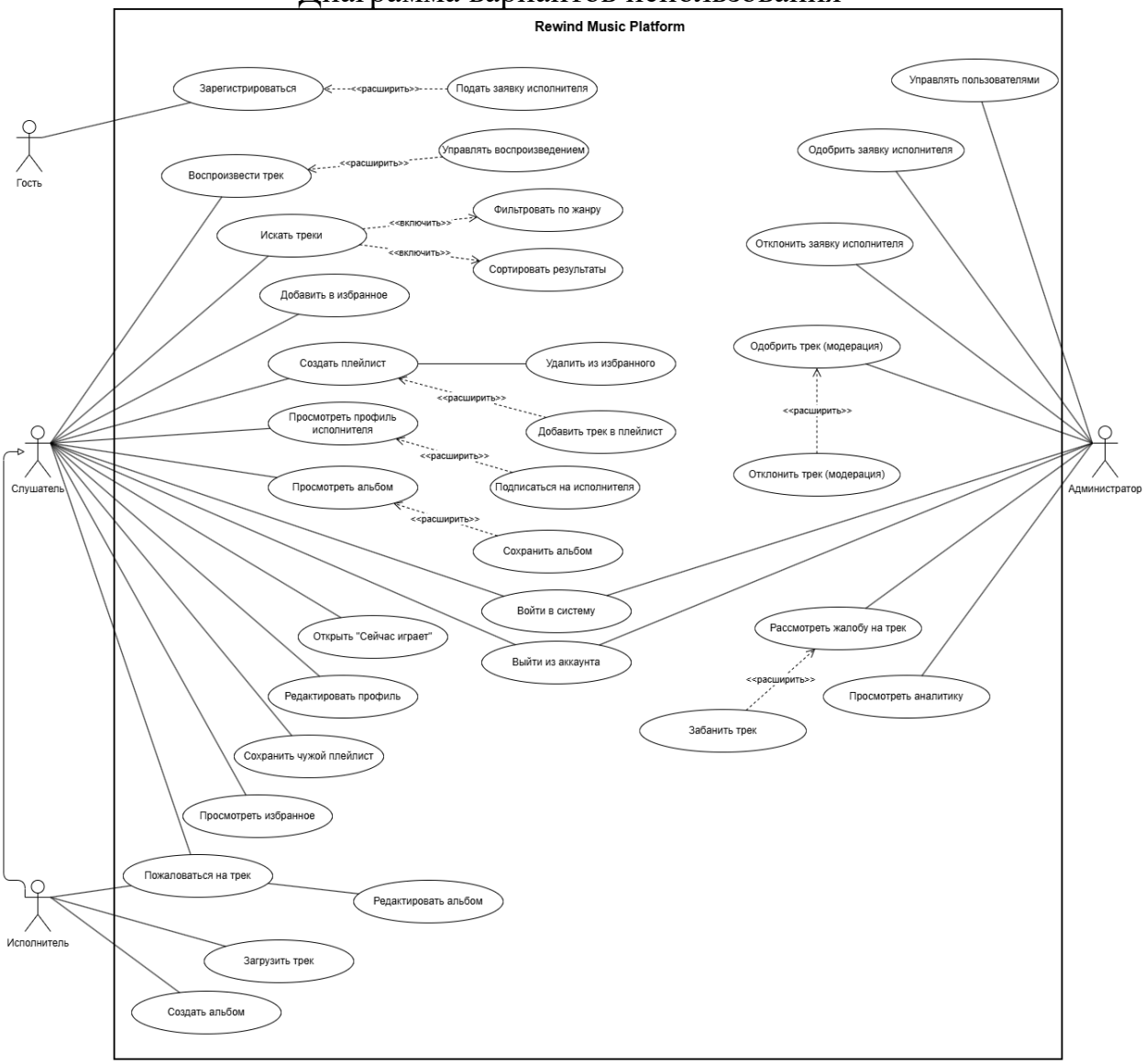
Таким образом, разработанное программное средство «Rewind» обеспечивает удобное воспроизведение и организацию музыкальных коллекций, соответствует всем заявленным требованиям и демонстрирует применение ключевых принципов объектно-ориентированного программирования и паттернов проектирования в рамках курса «Объектно-ориентированные технологии программирования и стандарты проектирования».

Список использованных источников

1. Microsoft Visual Studio [Электронный ресурс]. — Режим доступа: https://visualstudio.microsoft.com — Дата доступа: 05.05.2026
2. Полное руководство по языку программирования C# и платформе .NET [Электронный ресурс]. — Режим доступа: https://metanit.com/sharp/tutorial/ — Дата доступа: 05.05.2026
3. Пацей, Н.В. Курс лекций по языку программирования C# / Н.В. Пацей. — Минск: БГТУ, 2018. — 175 с.
4. Руководство по WPF [Электронный ресурс]. — Режим доступа: https://metanit.com/sharp/wpf/ — Дата доступа: 05.05.2026
5. WPF Documentation [Электронный ресурс]. — Режим доступа: https://learn.microsoft.com/ru-ru/dotnet/desktop/wpf/ — Дата доступа: 05.05.2026
6. Entity Framework Core Documentation [Электронный ресурс]. — Режим доступа: https://learn.microsoft.com/ru-ru/ef/core/ — Дата доступа: 05.05.2026
7. PostgreSQL 17 Documentation [Электронный ресурс]. — Режим доступа: https://www.postgresql.org/docs/17/ — Дата доступа: 05.05.2026
8. Блинова, Е.А. Курс лекций по базам данных / Е.А. Блинова. — Минск: БГТУ, 2019. — 175 с.
9. Npgsql — .NET Access to PostgreSQL [Электронный ресурс]. — Режим доступа: https://www.npgsql.org/ — Дата доступа: 05.05.2026
10. TagLib# (TagLibSharp) [Электронный ресурс]. — Режим доступа: https://github.com/mono/taglib-sharp — Дата доступа: 05.05.2026
11. Docker Documentation [Электронный ресурс]. — Режим доступа: https://docs.docker.com/ — Дата доступа: 05.05.2026
12. Figma Help Center [Электронный ресурс]. — Режим доступа: https://help.figma.com/ — Дата доступа: 05.05.2026

ПРИЛОЖЕНИЕ А

Диаграмма вариантов использования



ПРИЛОЖЕНИЕ Б

Листинг – XAML-код для страницы MainWindow

```
        <Window x:Class="Rewind.MainWindow"

xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation
"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-
compatibility/2006" xmlns:local="clr-namespace:Rewind"
        xmlns:controls="clr-namespace:Rewind.Contols"
        xmlns:help="clr-namespace:Rewind.Helpers"
        mc:Ignorable="d"
        Title="Rewind"
        Icon="{help:Icon rewind_logo.png}"
        Height="780" Width="1280"
        Background="{DynamicResource BgMain}">
    <Grid Background="{DynamicResource BgMain}">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="240"/>
            <ColumnDefinition Width="*"/>
        </Grid.ColumnDefinitions>
        <Grid.RowDefinitions>
            <RowDefinition Height="*"/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <!-- слайдер -->
        <Border Grid.Column="0" Grid.RowSpan="2"
                Background="{DynamicResource BgSidebar}"
                BorderBrush="{DynamicResource BorderColor}"
BorderThickness="0,0,1,0">
            <DockPanel LastChildFill="False">

                <!-- лого -->
                <StackPanel DockPanel.Dock="Top"
Margin="24,26,24,24">
                    <StackPanel Orientation="Horizontal">
                        <Border Width="32" Height="32"
CornerRadius="16" Margin="0,0,10,0">
                            <Image Source="{help:Icon
rewind_logo.png}" Stretch="UniformToFill"/></Border>
                        <TextBlock Text="Rewind"
VerticalAlignment="Center"
                                   FontWeight="Black"
FontSize="16" Foreground="{DynamicResource TextPrimary}"
                                   FontFamily="{StaticResource
HeaderFont}"/>
                    </StackPanel>
                </StackPanel>
```

```
                <!-- навигация -->
                <StackPanel DockPanel.Dock="Top"
Margin="12,0,12,0">
                    <TextBlock Text="МЕНЮ" FontSize="10"
FontWeight="Bold"
                               Foreground="{DynamicResource
TextMuted}" Margin="12,0,0,10"/>

                    <Button Style="{StaticResource
NavButtonStyle}" Click="ShowHome_Click" x:Name="BtnHome">
                        <StackPanel Orientation="Horizontal">
                            <Border Width="30" Height="30"
CornerRadius="7" Background="{DynamicResource BgMain}"
Margin="0,0,10,0">
                                <Rectangle x:Name="NavHomeIcon"
                                           Width="20"
                                           Height="20"
                                           Fill="{StaticResource
AccentColor}"

VerticalAlignment="Center"
                                           Cursor="Hand">
                                    <Rectangle.OpacityMask>
                                        <ImageBrush
ImageSource="{help:Icon nav_home.png}"

Stretch="Uniform"/>
                                    </Rectangle.OpacityMask>
                                </Rectangle>
                            </Border>
                            <TextBlock Text="Главная"
FontSize="14" Foreground="{DynamicResource TextPrimary}"
VerticalAlignment="Center"/>
                        </StackPanel>
                    </Button>

                    <Button Style="{StaticResource
NavButtonStyle}" x:Name="BtnSearch" Click="ShowSearch_Click">
                        <StackPanel Orientation="Horizontal">
                            <Border Width="30" Height="30"
CornerRadius="7" Background="{DynamicResource BgMain}"
Margin="0,0,10,0">
                                <Rectangle
x:Name="NavSearchIcon"
                                           Width="16"
                                           Height="16"
                                           Fill="{StaticResource
AccentColor}"

VerticalAlignment="Center"
                                           Cursor="Hand">
```

```
                                    <Rectangle.OpacityMask>
                                        <ImageBrush
ImageSource="{help:Icon nav_search.png}"

Stretch="Uniform"/>
                                    </Rectangle.OpacityMask>
                                </Rectangle>
                            </Border>
                            <TextBlock Text="Поиск"
FontSize="14" Foreground="{DynamicResource TextPrimary}"
VerticalAlignment="Center"/>
                        </StackPanel>
                    </Button>

                    <Button Style="{StaticResource
NavButtonStyle}" x:Name="BtnFavorites" Click="ShowLiked_Click">
                        <StackPanel Orientation="Horizontal">
                            <Border Width="30" Height="30"
CornerRadius="7" Background="{DynamicResource BgMain}"
Margin="0,0,10,0">
                                <Rectangle
x:Name="NavFavoritesIcon"
                                           Width="18"
                                           Height="18"
                                           Fill="{StaticResource
AccentColor}"

VerticalAlignment="Center"
                                           Cursor="Hand">
                                    <Rectangle.OpacityMask>
                                        <ImageBrush
ImageSource="{help:Icon nav_favorites.png}"

Stretch="Uniform"/>
                                    </Rectangle.OpacityMask>
                                </Rectangle>
                            </Border>
                            <TextBlock Text="Любимые"
FontSize="14" Foreground="{DynamicResource TextPrimary}"
VerticalAlignment="Center"/>
                        </StackPanel>
                    </Button>

                    <Button x:Name="BtnPlaylists"
Style="{StaticResource NavButtonStyle}"
Click="ShowPlaylists_Click">
                        <StackPanel Orientation="Horizontal">
                            <Border Width="30" Height="30"
CornerRadius="7" Background="{DynamicResource BgMain}"
Margin="0,0,10,0">
                                <Rectangle
x:Name="NavPlaylistsIcon"
```

```
                                                Width="18"
                                                Height="18"
                                                Fill="{StaticResource
AccentColor}"

VerticalAlignment="Center"
                                                Cursor="Hand">
                                            <Rectangle.OpacityMask>
                                                <ImageBrush
ImageSource="{help:Icon nav_playlists.png}"

Stretch="Uniform"/>
                                            </Rectangle.OpacityMask>
                                        </Rectangle>
                                    </Border>
                                    <TextBlock Text="Плейлисты"
FontSize="14" Foreground="{DynamicResource TextPrimary}"
VerticalAlignment="Center"/>
                                </StackPanel>
                            </Button>

                            <Button x:Name="BtnProfile"
Style="{DynamicResource ActiveNavButtonStyle}"
Click="ShowProfile_Click">
                                <StackPanel Orientation="Horizontal">
                                    <Border Width="30" Height="30"
CornerRadius="7" Background="{DynamicResource AccentColor}"
Margin="0,0,10,0">
                                        <Rectangle
x:Name="NavProfileIcon"
                                                   Width="18"
                                                   Height="18"

Fill="{DynamicResource AccentColor}"

VerticalAlignment="Center"
                                                   Cursor="Hand">
                                            <Rectangle.OpacityMask>
                                                <ImageBrush
ImageSource="{help:Icon nav_profile.png}"

Stretch="Uniform"/>
                                            </Rectangle.OpacityMask>
                                        </Rectangle>
                                    </Border>
                                    <TextBlock Text="Профиль"
FontSize="14" FontWeight="Bold"

Foreground="{DynamicResource AccentColor}"
VerticalAlignment="Center"/>
                                </StackPanel>
                            </Button>
```

```
                    <Button x:Name="BtnStudio"
Style="{StaticResource NavButtonStyle}" Click="ShowStudio_Click"
Visibility="Collapsed">
                        <StackPanel Orientation="Horizontal">
                            <Border Width="30" Height="30"
CornerRadius="7" Background="{DynamicResource BgMain}"
Margin="0,0,10,0">
                                <Rectangle
x:Name="NavStudioIcon"
                                           Width="18"
                                           Height="18"
                                           Fill="{StaticResource
AccentColor}"

VerticalAlignment="Center"
                                           Cursor="Hand">
                                    <Rectangle.OpacityMask>
                                        <ImageBrush
ImageSource="{help:Icon upload.png}" Stretch="Uniform"/>
                                    </Rectangle.OpacityMask>
                                </Rectangle>
                            </Border>
                            <TextBlock Text="Студия"
FontSize="13" Foreground="{DynamicResource TextPrimary}"
VerticalAlignment="Center"/>
                        </StackPanel>
                    </Button>
                </StackPanel>
            </DockPanel>
        </Border>
        <ContentControl x:Name="MainContentArea" Grid.Column="1"
Grid.Row="0" />
        <controls:PlayerBar x:Name="GlobalPlayerBar"
                            Grid.Column="0" Grid.Row="1"
                            Grid.ColumnSpan="2"
                            Height="76"
                            Visibility="Collapsed"
                            Panel.ZIndex="50"/>

        <Border x:Name="ToastBorder"
                Grid.Column="1" Grid.Row="0"
                VerticalAlignment="Bottom"
HorizontalAlignment="Right"
                Margin="0,0,24,110"
                CornerRadius="16" Padding="18,14"
                MinWidth="280" MaxWidth="340"
                Visibility="Collapsed"
                Panel.ZIndex="300">
            <Border.Background>
                <SolidColorBrush Color="#1F1F1D"/>
            </Border.Background>
```

```
            <Border.Effect>
                <DropShadowEffect BlurRadius="24"
ShadowDepth="4"
                                  Color="#000000"
Opacity="0.4"/>
            </Border.Effect>
            <Grid>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="Auto"/>
                    <ColumnDefinition Width="*"/>
                    <ColumnDefinition Width="Auto"/>
                </Grid.ColumnDefinitions>
                <Image Grid.Column="0" Source="{help:Icon
music_note.png}"
                       Width="22" Height="22"
VerticalAlignment="Center"
                       Margin="0,0,10,0"/>
                <StackPanel Grid.Column="1"
VerticalAlignment="Center">
                    <TextBlock x:Name="ToastTitle"
                               FontSize="13" FontWeight="Bold"
Foreground="White"
                               FontFamily="{StaticResource
HeaderFont}"
                               TextWrapping="Wrap"
Margin="0,0,0,4"/>
                    <TextBlock x:Name="ToastBody"
                               FontSize="12"
Foreground="#AAAAAA"
                               FontFamily="{StaticResource
HeaderFont}"
                               TextWrapping="Wrap"/>
                </StackPanel>
                <Border Grid.Column="2" CornerRadius="10"
                        Background="#2D2D2B" Padding="8,5"
                        Margin="12,0,0,0"
VerticalAlignment="Top"
                        Cursor="Hand"

MouseLeftButtonDown="ToastDismiss_Click">
                    <TextBlock Text="✕" FontSize="10"
Foreground="#888880"
                               HorizontalAlignment="Center"
VerticalAlignment="Center"/>
                </Border>
            </Grid>
        </Border>
    </Grid>
</Window>
```

ПРИЛОЖЕНИЕ В

Листинг – ViewModel для AlbumViewModel

```
using Rewind.Helpers;
using Rewind.MVVM.Core;
using Rewind.MVVM.Services;
using System.Windows.Input;

namespace Rewind.MVVM.ViewModels.Entities
{
    public class AlbumViewModel : ObservableObject
    {
        public Album Model { get; }

        public AlbumViewModel(Album album)
        {
            Model = album ?? throw new
ArgumentNullException(nameof(album));
            OpenCommand = new RelayCommand(Open);
        }

        public int AlbumId => Model.AlbumId;
        public string Title => Model.Title;
        public string? CoverPath => Model.CoverPath;
        public int TrackCount => Model.AlbumTracks?.Count ?? 0;

        public string ArtistName =>
            Model.Artist?.Nickname
            ?? UserService.GetUserById(Model.ArtistId)?.Nickname
            ?? "Исполнитель";

        public string Subtitle => $"{ArtistName} • {TrackCount}
треков";

        public ICommand OpenCommand { get; }

        private void Open()
        {
            var nav =
ServiceLocator.TryResolve<INavigationService>();
            var full = AlbumService.GetById(AlbumId) ?? Model;
            nav?.OpenAlbumDetails(full);
        }
    }
}
```

ПРИЛОЖЕНИЕ Г

Листинг – исходный код класса AppDbContext

```
using Microsoft.EntityFrameworkCore;

namespace Rewind.Helpers
{
    public class AppDbContext : DbContext
    {
        private static volatile bool _schemaInitialized;
        private static readonly object _initLock = new();

        public AppDbContext()
        {
            if (_schemaInitialized) return;
            lock (_initLock)
            {
                if (_schemaInitialized) return;
                Database.EnsureCreated();
                EnsureArtistRequestsTable();
                EnsureTrackSchemaUpdates();
                EnsureAlbumsTables();
                EnsureSubscriptionTimestamp();
                SeedDefaultAdmin();
                _schemaInitialized = true;
            }
        }

        private void EnsureTrackSchemaUpdates()
        {
            try { Database.ExecuteSqlRaw(@"ALTER TABLE
""Tracks"" ADD COLUMN IF NOT EXISTS ""Genre"" VARCHAR(100)"); }
catch { }
            try { Database.ExecuteSqlRaw(@"ALTER TABLE
""Tracks"" ADD COLUMN IF NOT EXISTS ""PublishStatus""
VARCHAR(20) NOT NULL DEFAULT 'Published'"); } catch { }
            try { Database.ExecuteSqlRaw(@"ALTER TABLE
""Tracks"" ADD COLUMN IF NOT EXISTS ""RejectionReason"" TEXT");
} catch { }
        }

        private void EnsureSubscriptionTimestamp()
        {
            try
            {
                Database.ExecuteSqlRaw(@"
                    ALTER TABLE ""Subscriptions""
                    ADD COLUMN IF NOT EXISTS ""SubscribedAt""
                    TIMESTAMPTZ NOT NULL DEFAULT NOW()");
            }
            catch
```

```
            {

            }
        }

        private void EnsureAlbumsTables()
        {
            try
            {
                Database.ExecuteSqlRaw(@"
                    CREATE TABLE IF NOT EXISTS ""Albums"" (
                        ""AlbumId""    SERIAL       PRIMARY KEY,
                        ""Title""      VARCHAR(100) NOT NULL,
                        ""ArtistId""   INT          NOT NULL
REFERENCES ""Users""(""UserId"") ON DELETE CASCADE,
                        ""CoverPath""  TEXT,
                        ""Genre""      VARCHAR(100),
                        ""CreatedAt""  TIMESTAMPTZ  NOT NULL
DEFAULT NOW()
                    )");
                Database.ExecuteSqlRaw(@"
                    CREATE TABLE IF NOT EXISTS ""AlbumTracks"" (
                        ""AlbumId""  INT NOT NULL REFERENCES
""Albums""(""AlbumId"") ON DELETE CASCADE,
                        ""TrackId""  INT NOT NULL REFERENCES
""Tracks""(""TrackID"") ON DELETE CASCADE,
                        PRIMARY KEY (""AlbumId"", ""TrackId"")
                    )");
                Database.ExecuteSqlRaw(@"
                    CREATE TABLE IF NOT EXISTS ""SavedAlbums"" (
                        ""UserId"" INT NOT NULL REFERENCES
""Users""(""UserId"") ON DELETE CASCADE,
                        ""AlbumId"" INT NOT NULL REFERENCES
""Albums""(""AlbumId"") ON DELETE CASCADE,
                        ""SavedAt"" TIMESTAMPTZ NOT NULL DEFAULT
NOW(),
                        PRIMARY KEY (""UserId"", ""AlbumId"")
                    )");
                Database.ExecuteSqlRaw(@"
                    CREATE TABLE IF NOT EXISTS
""AlbumListenEvents"" (
                        ""AlbumListenEventId"" SERIAL PRIMARY
KEY,
                        ""UserId"" INT NOT NULL REFERENCES
""Users""(""UserId"") ON DELETE CASCADE,
                        ""AlbumId"" INT NOT NULL REFERENCES
""Albums""(""AlbumId"") ON DELETE CASCADE,
                        ""ListenedAt"" TIMESTAMPTZ NOT NULL
DEFAULT NOW()
                    )");
                Database.ExecuteSqlRaw(@"
```

```
                    CREATE TABLE IF NOT EXISTS ""TrackReports""
(
                        ""ReportId""    SERIAL       PRIMARY
KEY,
                        ""TrackId""     INT          NOT NULL
REFERENCES ""Tracks""(""TrackID"") ON DELETE CASCADE,
                        ""ReporterId""  INT          NOT NULL
REFERENCES ""Users""(""UserId"") ON DELETE CASCADE,
                        ""Reason""      TEXT         NOT NULL,
                        ""Status""      VARCHAR(20)  NOT NULL
DEFAULT 'Pending',
                        ""CreatedAt""   TIMESTAMPTZ  NOT NULL
DEFAULT NOW()
                    )");
                Database.ExecuteSqlRaw(@"
                    CREATE TABLE IF NOT EXISTS
""SavedPlaylists"" (
                        ""UserId""      INT         NOT NULL
REFERENCES ""Users""(""UserId"") ON DELETE CASCADE,
                        ""PlaylistId""  INT         NOT NULL
REFERENCES ""Playlists""(""PlaylistID"") ON DELETE CASCADE,
                        ""SavedAt""     TIMESTAMPTZ NOT NULL
DEFAULT NOW(),
                        PRIMARY KEY (""UserId"", ""PlaylistId"")
                    )");
                Database.ExecuteSqlRaw(@"
                    CREATE TABLE IF NOT EXISTS
""PlaylistListens"" (
                        ""UserId""     INT         NOT NULL
REFERENCES ""Users""(""UserId"") ON DELETE CASCADE,
                        ""PlaylistId"" INT         NOT NULL
REFERENCES ""Playlists""(""PlaylistID"") ON DELETE CASCADE,
                        ""ListenedAt"" TIMESTAMPTZ NOT NULL
DEFAULT NOW(),
                        PRIMARY KEY (""UserId"", ""PlaylistId"")
                    )");
                Database.ExecuteSqlRaw(@"
                    CREATE TABLE IF NOT EXISTS
""PlaylistPlayEvents"" (
                        ""PlaylistPlayEventId"" SERIAL PRIMARY
KEY,
                        ""UserId""     INT         NOT NULL
REFERENCES ""Users""(""UserId"") ON DELETE CASCADE,
                        ""PlaylistId"" INT         NOT NULL
REFERENCES ""Playlists""(""PlaylistID"") ON DELETE CASCADE,
                        ""ListenedAt"" TIMESTAMPTZ NOT NULL
DEFAULT NOW()
                    )");
            }
            catch { }
        }
```

```
        private void EnsureArtistRequestsTable()
        {
            try
            {
                Database.ExecuteSqlRaw(@"
                    CREATE TABLE IF NOT EXISTS
""ArtistRequests"" (
                        ""RequestId""     SERIAL
PRIMARY KEY,
                        ""Nickname""      VARCHAR(50)     NOT
NULL,
                        ""Email""         VARCHAR(100)    NOT
NULL,
                        ""PasswordHash""  TEXT            NOT
NULL,
                        ""Status""        VARCHAR(20)     NOT
NULL DEFAULT 'Pending',
                        ""CreatedAt""     TIMESTAMPTZ     NOT
NULL DEFAULT NOW()
                    )");
            }
            catch
            {

            }
        }
        public DbSet<Role> Roles { get; set; }
        public DbSet<User> Users { get; set; }
        public DbSet<ArtistRequest> ArtistRequests { get; set; }
        public DbSet<Album> Albums { get; set; }
        public DbSet<AlbumTrack> AlbumTracks { get; set; }
        public DbSet<SavedAlbum> SavedAlbums { get; set; }
        public DbSet<AlbumListenEvent> AlbumListenEvents { get;
set; }
        public DbSet<TrackReport> TrackReports { get; set; }
        public DbSet<SavedPlaylist> SavedPlaylists { get; set; }
        public DbSet<PlaylistListen> PlaylistListens { get; set;
}
        public DbSet<PlaylistPlayEvent> PlaylistPlayEvents {
get; set; }
        public DbSet<Track> Tracks { get; set; }
        public DbSet<Statistic> Statistics { get; set; }
        public DbSet<Playlist> Playlists { get; set; }
        public DbSet<PlaylistTrack> PlaylistTracks { get; set; }
        public DbSet<Subscription> Subscriptions { get; set; }
        public DbSet<Favorite> Favorites { get; set; }
        public DbSet<ListeningHistory> History { get; set; }

        protected override void
OnConfiguring(DbContextOptionsBuilder o)
```

```
            =>
o.UseNpgsql("Host=localhost;Port=5433;Database=rewinddb;Username
=postgres;Password=5329965");

        protected override void OnModelCreating(ModelBuilder m)
        {
            m.Entity<User>().HasIndex(u =>
u.Nickname).IsUnique();
            m.Entity<User>().HasIndex(u => u.Email).IsUnique();

            m.Entity<PlaylistTrack>().HasKey(pt => new {
pt.PlaylistID, pt.TrackID });
            m.Entity<Favorite>().HasKey(f => new { f.UserID,
f.TrackID });
            m.Entity<Subscription>().HasKey(s => new {
s.FollowerID, s.ArtistID });
            m.Entity<AlbumTrack>().HasKey(at => new {
at.AlbumId, at.TrackId });
            m.Entity<SavedAlbum>().HasKey(sa => new { sa.UserId,
sa.AlbumId });
            m.Entity<SavedPlaylist>().HasKey(sp => new {
sp.UserId, sp.PlaylistId });
            m.Entity<PlaylistListen>().HasKey(pl => new {
pl.UserId, pl.PlaylistId });

            m.Entity<ListeningHistory>().HasIndex(h =>
h.UserID);
            m.Entity<ListeningHistory>().HasIndex(h =>
h.TrackID);

            m.Entity<Role>().HasData(
                new Role { RoleId = 1, RoleName = "Admin" },
                new Role { RoleId = 2, RoleName = "Artist" },
                new Role { RoleId = 3, RoleName = "Listener" }
            );
        }

        private void SeedDefaultAdmin()
        {
            try
            {
                if (Users.Any(u => u.Nickname == "Alexey"))
return;

                Users.Add(new User
                {
                    Nickname = "Alexey",
                    Email = "alexey@rewind.local",
                    PasswordHash =
PasswordHelper.HashPassword("20062018no"),
                    ProfilePhotoPath = null,
                    RoleId = 1,
```

```
                    Status = "Активен"
                });
                SaveChanges();
            }
            catch
            {
            }
        }
    }
}
```

ПРИЛОЖЕНИЕ Д

Листинг – класс FileStorage

```
using System.IO;

namespace Rewind.Helpers
{
    public static class FileStorage
    {
        public const string DefaultAvatar =
"Images/Avatars/default_avatar.png";

        public static string CopyAvatar(string sourcePath) =>
CopyFile(sourcePath, "Images/Avatars", keepOriginalName: true);
        public static string CopyTrackCover(string sourcePath)
=> CopyFile(sourcePath, "Images/TrackCovers", keepOriginalName:
true);
        public static string CopyPlaylistCover(string
sourcePath) => CopyFile(sourcePath, "Images/PlaylistCovers",
keepOriginalName: true);
        public static string CopyAlbumCover(string sourcePath)
=> CopyFile(sourcePath, "Images/AlbumCovers", keepOriginalName:
true);
        public static string CopyTrackAudio(string sourcePath,
string trackName)
        {
            string extension = Path.GetExtension(sourcePath);
            string safe = SanitizeFileName(trackName);
            if (string.IsNullOrWhiteSpace(safe)) safe = "track";

            string relative = CopyFile(sourcePath,
"MusicLibrary", keepOriginalName: false, forcedBaseName: safe,
forcedExtension: extension);
            return Path.GetFileName(relative);
        }

        public static string ResolvePath(string? storedPath)
        {
            if (string.IsNullOrWhiteSpace(storedPath)) return
string.Empty;
            if (Path.IsPathRooted(storedPath)) return
storedPath;
            return
Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
storedPath.Replace('/', Path.DirectorySeparatorChar));
        }

        public static string ResolveImagePath(string?
storedPath, string legacyFolder = "CoversLibrary")
        {
```

```
            if (string.IsNullOrWhiteSpace(storedPath)) return
string.Empty;
            if (Path.IsPathRooted(storedPath)) return
storedPath;

            string direct = ResolvePath(storedPath);
            if (File.Exists(direct)) return direct;

            string legacy =
Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
legacyFolder, storedPath);
            return legacy;
        }

        private static string CopyFile(string sourcePath, string
relativeFolder, bool keepOriginalName, string? forcedBaseName =
null, string? forcedExtension = null)
        {
            string folder =
Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
relativeFolder.Replace('/', Path.DirectorySeparatorChar));
            Directory.CreateDirectory(folder);

            string ext = forcedExtension ??
Path.GetExtension(sourcePath);
            string baseName = forcedBaseName ??
(keepOriginalName ? Path.GetFileNameWithoutExtension(sourcePath)
: Guid.NewGuid().ToString());
            baseName = SanitizeFileName(baseName);
            if (string.IsNullOrWhiteSpace(baseName)) baseName =
"file";

            string fileName = $"{baseName}{ext}";
            string dest = Path.Combine(folder, fileName);
            int n = 1;
            while (File.Exists(dest))
            {
                fileName = $"{baseName}_{n++}{ext}";
                dest = Path.Combine(folder, fileName);
            }
            File.Copy(sourcePath, dest, overwrite: false);
            return Path.Combine(relativeFolder,
fileName).Replace('\\', '/');
        }
        public static string SanitizeFileName(string name)
        {
            var invalid = Path.GetInvalidFileNameChars();
            return new string(name.Select(ch =>
invalid.Contains(ch) ? '_' : ch).ToArray()).Trim();
        }
    }
}
```

ПРИЛОЖЕНИЕ Е

Листинг – класс RegistrationViewModel

```
using System.Windows;
using System.Windows.Controls;
using Rewind.MVVM.Services;
using Rewind.MVVM.ViewModels.Pages;

namespace Rewind.Pages
{
    public partial class Registration : Window
    {
        private readonly RegistrationViewModel _vm;

        public Registration()
        {
            InitializeComponent();
            _vm = new
RegistrationViewModel(ServiceLocator.Resolve<IDialogService>());
            DataContext = _vm;
            _vm.PropertyChanged += (_, _) => UpdateUI();
            UpdateUI();
        }

        private void RegisterButton_Click(object sender,
RoutedEventArgs e)
        {
            switch (_vm.SelectedRoleId)
            {
                case 1:
                    _vm.Email = UserRegEmail.Text;
                    _vm.Nickname = UserRegNickname.Text;
                    _vm.Password = UserRegPassword.Password;
                    _vm.ConfirmPassword =
UserRegPasswordConfirm.Password;
                    break;
                case 2:
                    _vm.Email = ArtistRegEmail.Text;
                    _vm.Nickname = ArtistRegNickname.Text;
                    _vm.Password = ArtistRegPassword.Password;
                    _vm.ConfirmPassword =
ArtistRegPasswordConfirm.Password;
                    break;
            }
            _vm.RegisterCommand.Execute(null);
        }

        private void LoginButton_Click(object sender,
RoutedEventArgs e)
        {
            switch (_vm.SelectedRoleId)
```

```
            {
                case 1:
                    _vm.Login = UserLoginValue.Text;
                    _vm.Password = UserPasswordValue.Password;
                    break;
                case 2:
                    _vm.Login = ArtistLoginValue.Text;
                    _vm.Password = ArtistPasswordValue.Password;
                    break;
                case 3:
                    _vm.Login = AdminLoginValue.Text;
                    _vm.Password = AdminPasswordValue.Password;
                    break;
            }
            _vm.LoginCommand.Execute(null);
        }


        private void RoleSelector_SelectionChanged(object
sender, SelectionChangedEventArgs e)
        {
            if (RoleSelector == null || PanelUser == null)
return;
            _vm.SelectedRoleId = RoleSelector.SelectedIndex + 1;
        }

        private void ModeBtn_Click(object sender,
RoutedEventArgs e)
        {
            if (sender is not Button btn) return;
            if (btn.Name == "LoginModeBtn")
_vm.SwitchToLoginCommand.Execute(null);
            else _vm.SwitchToRegisterCommand.Execute(null);
        }

        private void UpdateUI()
        {
            if (LoginModeBtn == null || RegModeBtn == null)
return;

            LoginModeBtn.Style =
(Style)FindResource(_vm.IsLoginMode ? "ModeBtnActive" :
"ModeBtn");
            RegModeBtn.Style =
(Style)FindResource(!_vm.IsLoginMode ? "ModeBtnActive" :
"ModeBtn");

            if (_vm.SelectedRoleId == 3)
            {
                RegModeBtn.Style =
(Style)FindResource("ModeBtnActive");
                RegModeBtn.Visibility = Visibility.Collapsed;
```

```
                Grid.SetColumnSpan(LoginModeBtn, 2);
            }
            else
            {
                RegModeBtn.Visibility = Visibility.Visible;
                Grid.SetColumnSpan(LoginModeBtn, 1);
            }

            PanelUser.Visibility = Visibility.Collapsed;
            PanelArtist.Visibility = Visibility.Collapsed;
            PanelAdmin.Visibility = Visibility.Collapsed;
            UserLoginForm.Visibility = Visibility.Collapsed;
            UserRegForm.Visibility = Visibility.Collapsed;
            ArtistLoginForm.Visibility = Visibility.Collapsed;
            ArtistRegForm.Visibility = Visibility.Collapsed;

            switch (_vm.SelectedRoleId)
            {
                case 1:
                    PanelUser.Visibility = Visibility.Visible;
                    (_vm.IsLoginMode ? UserLoginForm :
(FrameworkElement)UserRegForm).Visibility = Visibility.Visible;
                    break;
                case 2:
                    PanelArtist.Visibility = Visibility.Visible;
                    (_vm.IsLoginMode ? ArtistLoginForm :
(FrameworkElement)ArtistRegForm).Visibility =
Visibility.Visible;
                    break;
                case 3:
                    PanelAdmin.Visibility = Visibility.Visible;
                    break;
            }
        }
    }
}
```

ПРИЛОЖЕНИЕ Ж

Листинг – ViewModel для ArtistStudioViewModel

```
using Rewind.Helpers;
using Rewind.MVVM.Core;
using Rewind.MVVM.Services;
using System.IO;
using System.Windows.Input;

namespace Rewind.MVVM.ViewModels.Pages
{
    public sealed class ArtistStudioViewModel : ViewModelBase
    {
        private readonly IDialogService _dialog;

        public ArtistStudioViewModel(IDialogService dialog)
        {
            _dialog = dialog;
            UploadTrackCommand = new RelayCommand(UploadTrack);
            CreateAlbumCommand = new RelayCommand(CreateAlbum);
            SelectTabCommand = new RelayCommand<string>(name =>
{ if (name != null) ActiveTab = name; });
        }


        private string _activeTab = "upload";
        public string ActiveTab
        {
            get => _activeTab;
            set { if (SetProperty(ref _activeTab, value))
TabChanged?.Invoke(); }
        }
        public event Action? TabChanged;


        public string NewTrackTitle { get; set; } = "";
        public string NewTrackGenre { get; set; } = "";
        public string? NewTrackCoverPath { get; set; }
        public string? NewTrackAudioPath { get; set; }
        public event Action? UploadFinished;

        public ICommand UploadTrackCommand { get; }
        public ICommand CreateAlbumCommand { get; }
        public ICommand SelectTabCommand { get; }


        public string AlbumName { get; set; } = "";
        public string AlbumGenre { get; set; } = "";
        public string? AlbumCoverPath { get; set; }
        public List<int> AlbumSelectedTrackIds { get; set; } =
new();
```

```
        public int? EditingAlbumId { get; set; }
        public event Action? AlbumSaved;

        private void UploadTrack()
        {
            if (string.IsNullOrWhiteSpace(NewTrackTitle) ||
string.IsNullOrEmpty(NewTrackAudioPath))
            { _dialog.Error("Введите название и выберите
аудиофайл!"); return; }

            try
            {
                string fileName =
FileStorage.CopyTrackAudio(NewTrackAudioPath, NewTrackTitle);
                string destPath =
Path.Combine(AppDomain.CurrentDomain.BaseDirectory,
"MusicLibrary", fileName);
                int duration;
                using (var f = TagLib.File.Create(destPath))
                    duration =
(int)f.Properties.Duration.TotalSeconds;

                string? coverPath =
!string.IsNullOrWhiteSpace(NewTrackCoverPath)
                    ?
FileStorage.CopyTrackCover(NewTrackCoverPath) : null;

                var track = new Track
                {
                    Title = NewTrackTitle,
                    FilePath = fileName,
                    CoverPath = coverPath,
                    Duration = duration,
                    UploadDate = DateTime.UtcNow,
                    ArtistID = Session.UserId,
                    Genre = NewTrackGenre,
                    PublishStatus = "Pending"
                };
                TrackService.AddTrack(track);
                _dialog.Info("Трек отправлен на проверку
администратору.", "Заявка отправлена");

                NewTrackTitle = ""; NewTrackAudioPath = null;
NewTrackCoverPath = null;
                UploadFinished?.Invoke();
            }
            catch (Exception ex) { _dialog.Error(ex.Message); }
        }

        private void CreateAlbum()
        {
```

```
            if (string.IsNullOrWhiteSpace(AlbumName)) {
_dialog.Error("Введите название альбома"); return; }
            if (AlbumSelectedTrackIds.Count == 0) {
_dialog.Error("Выберите хотя бы один трек"); return; }

            try
            {
                int albumId;
                if (EditingAlbumId.HasValue)
                {
                    AlbumService.Update(EditingAlbumId.Value,
AlbumName, AlbumGenre, AlbumCoverPath);
                    albumId = EditingAlbumId.Value;
                }
                else
                {
                    albumId = AlbumService.Create(AlbumName,
Session.UserId, AlbumGenre, AlbumCoverPath);
                }
                foreach (var trackId in AlbumSelectedTrackIds)
                    AlbumService.AddTrack(albumId, trackId);

                _dialog.Info(EditingAlbumId.HasValue ? "Альбом
обновлён." : "Альбом создан.");

                AlbumName = ""; AlbumGenre = ""; AlbumCoverPath
= null;
                AlbumSelectedTrackIds = new List<int>();
EditingAlbumId = null;
                AlbumSaved?.Invoke();
            }
            catch (Exception ex) { _dialog.Error(ex.Message); }
        }
    }
}
```